

《星溯旅人》游戏策划书

部分内容由豆包生成

 **核心概念：**一款科幻题材的2.5D箱庭探索策略RPG，玩家扮演星际探险家，在浩瀚宇宙中探索精心设计的星球关卡、揭开古老文明的秘密。

一、游戏概述

1.1 基本信息

项目	内容
游戏名称	星溯旅人 (Star Retrospect Traveler)
游戏类型	2.5D箱庭探索 · 策略冒险 · RPG
游戏平台	PC (Steam/Epic) 、PS5、Xbox Series X/S
目标用户	18-35岁，喜欢科幻题材、探索类游戏的核心玩家
游戏引擎	Qt 6.10.1
预计开发周期	24个月
游戏分级	ESRB: T / PEGI: 12

1.2 核心卖点

- 时间褶皱探索：**进入先驱者遗迹的时间褶皱，在过去与现实间穿梭，解开跨越时空的谜题
- 星核共鸣系统：**与每颗星球的星核产生灵能共鸣，获得操控环境的独特能力，但需承受灵能反噬风险
- 生态共生玩法：**与异星生物建立共生关系，获得飞行、水下呼吸等特殊能力，你的选择影响整个星球生态平衡
- 因果叙事分支：**你的选择不仅影响现在，还会通过时间褶皱改变过去，形成独特的多时间线剧情体验
- 动态世界演化：**星球生态、势力格局会随玩家行为动态演化，每个周目都有截然不同的游戏体验

1.3 游戏定位

《星溯旅人》定位为2.5D箱庭探索策略RPG游戏，主打"时空解谜"与"因果叙事"。游戏基于Qt原生库开发，以时间褶皱、星核共鸣、生态共生三大创新系统为核心，采用关卡式星球探索结构，让玩家在精心设计的箱庭场景中体验独特的时空冒险，同时通过多时间线的剧情设计，提供深度沉浸式的叙事体验。

二、世界观设定

2.1 背景故事

公元3247年，人类已在银河系建立了庞大的星际联邦。然而，一场被称为"大坍缩"的神秘灾难摧毁了联邦的核心区域，数百颗星球失联，古老的星门网络陷入瘫痪。

玩家扮演一名年轻的星际探险家，在一次例行任务中意外发现了一块来自远古文明"先驱者"的神秘碎片。这块碎片似乎与"大坍缩"的真相有着千丝万缕的联系。为了揭开谜团，玩家驾驶着自己的飞船，踏上了穿越星渊的冒险之旅。

2.2 主要势力

势力名称	特点	与玩家关系
星际联邦	残存的人类政府，致力于恢复秩序	初始友好，可结盟或对立
自由贸易联盟	由商人和雇佣兵组成的松散组织	中立，可交易与接取任务
虚空教团	崇拜"大坍缩"的神秘宗教组织	主要敌对势力
先驱者遗族	远古文明的后裔，守护着古老秘密	神秘中立，关键剧情NPC

2.3 核心设定

- 星门网络**：远古文明遗留的超空间传送装置，是星际旅行的关键
- 灵能**：部分人类觉醒的超自然能力，与先驱者文明有关
- 大坍缩**：20年前发生的神秘灾难，原因不明，是游戏核心悬念
- 先驱者**：已经消亡的远古文明，留下了无数遗迹与谜团
- 时间褶皱**：先驱者文明实验产生的时空异常，可进入不同时间线，是大坍缩的成因之一

- 星核：每颗星球核心的先驱者能量结晶，可通过灵能共鸣获得操控环境的能力
- 生态灵能：星球生物体内蕴含的灵能，与生物建立共生关系可共享其特殊能力

三、核心玩法

3.0 游戏核心框架

《星溯旅人》采用"探索-解谜-战斗-成长"的核心循环，以三大创新系统为核心，构建一个富有深度和策略性的游戏框架。

3.0.1 核心循环

探索

- 探索星球箱庭关卡
- 发现时间褶皱入口
- 寻找星核与共生生物
- 收集资源与装备

解谜

- 时间褶皱时空解谜
- 星核共鸣环境谜题
- 生态共生协作谜题
- 多系统联动解谜

战斗

- 策略性战斗遭遇
- 运用三大系统能力
- 共生伙伴协同作战
- BOSS挑战战

成长

- 获得经验与装备
- 解锁新天赋能力
- 提升共鸣与共生等级
- 解锁新区域与内容

 **核心循环特色：**每个循环都会让玩家变得更强大，同时解锁更多探索内容，形成正向反馈循环。三大创新系统贯穿整个循环，提供丰富的策略选择。

3.0.2 游戏整体流程

- 启动阶段：游戏启动 → 厂商logo → 主菜单 → 开始/继续游戏
- 新手阶段：新伊甸星球教学 → 基础操作学习 → 时间褶皱系统解锁 → 第一个谜题
- 探索阶段：解锁飞船 → 前往不同星球 → 逐步解锁星核共鸣与生态共生系统
- 成长阶段：完成任务与挑战 → 收集资源与装备 → 升级天赋与能力
- 剧情阶段：推进主线剧情 → 揭示大坍缩真相 → 做出关键选择
- 终局阶段：最终BOSS战 → 结局呈现 → 制作人员名单
- 通关后：新游戏+解锁 → 多周目体验 → 隐藏内容探索 → 成就收集

3.0.3 游戏节奏设计

阶段	时长	节奏特点

新手期	1-3小时	快速上手，频繁正反馈，每10-15分钟有新内容解锁
成长期	5-15小时	系统逐步解锁，探索深度增加，每30-60分钟有重大进展
成熟期	10-20小时	全部系统解锁，挑战难度提升，每1-2小时有剧情高潮
终局期	3-5小时	节奏加快，连续高潮，最终BOSS战与结局呈现

3.0.4 系统关系框架

- **核心层：**时间褶皱系统、星核共鸣系统、生态共生系统（三大创新系统，相互联动）
- **战斗层：**战斗系统（运用三大系统能力进行策略战斗）
- **成长层：**天赋系统、装备系统、数值系统（支撑玩家成长）
- **内容层：**任务系统、关卡设计、剧情叙事（提供游戏内容）
- **体验层：**成就系统、存档系统、难度系统（保障玩家体验）

 **系统联动特色：**三大核心系统不是孤立的，而是深度联动。时间褶皱可以改变星核状态，星核共鸣可以影响生态环境，生态共生可以帮助穿越时间褶皱，形成"三位一体"的深度玩法体验。

3.0.5 玩家成长路径

 新手期（1-10级）	 成长期（11-30级）	 成熟期（31-50级）
● 解锁：基础操作、时间褶皱系统 ● 获得：初始装备、第一个共生伙伴 ● 体验：新手星球探索、教学谜题 ● 目标：熟悉游戏，建立基础认知	● 解锁：星核共鸣系统、生态共生系统 ● 获得：更多装备、天赋树解锁、多个共生伙伴 ● 体验：多星球探索、系统联动解谜 ● 目标：掌握三大系统，形成个人玩法风格	● 解锁：全部系统、终极联动能力 ● 获得：顶级装备、完整天赋、传说共生伙伴 ● 体验：高难度挑战、深度剧情、隐藏内容 ● 目标：挑战终极内容，探索所有可能性

3.0.6 经济循环设计

- **主要货币：**星际信用点（通用货币，用于购买装备、道具、升级）
- **特殊货币：**时间碎片（稀有货币，用于解锁特殊能力和时间相关内容）

- **产出途径：**任务奖励、战斗掉落、探索收集、出售物品
- **消耗途径：**购买装备、购买道具、装备升级、天赋解锁、商店消费
- **回收系统：**装备分解、物品出售、商店回购、货币兑换

 **经济平衡原则：**确保货币产出与消耗保持动态平衡，避免通货膨胀或货币短缺。不同难度下货币产出有所调整，高难度获得更多奖励。

3.1 时间褶皱系统

时间褶皱系统的核心是**时间织网**机制——时间不是线性的河流，而是一张可以编织、扭曲、折叠的网。玩家不仅可以在不同时间线间穿梭，更能主动创造时间分支、收集悖论能量、甚至召唤不同时间点的自己协同作战。时间不再只是背景设定，而是玩家可以主动操控的核心玩具。

时间褶皱机制

- **多层时间线：**每个遗迹存在3-5层时间线，对应星球的不同历史时期（繁荣期、灾难期、衰败期等）
- **时空切换：**玩家可在特定节点切换时间线，同一地点在不同时间线呈现完全不同的面貌
- **时间悖论：**在过去做出的改变会影响现在的世界，形成独特的因果循环玩法
- **灵能消耗：**进入和停留在时间褶皱中会持续消耗灵能，灵能耗尽会被强制弹出

时间褶皱类型

- **稳定褶皱：**时间流速稳定，可长时间停留，适合探索和解谜
- **不稳定褶皱：**时间流速随机变化，可能突然加速或减速，风险高但奖励丰厚
- **断裂褶皱：**时间线破碎成多个碎片，需要修复后才能完整探索
- **镜像褶皱：**存在正反两个时间线，一个正向流动一个逆向流动，需在两个时间线间切换解谜
- **循环褶皱：**时间不断循环重置，玩家需要在循环中找到打破循环的方法

时间线特色

- **繁荣期：**先驱者文明鼎盛时期，城市繁华、科技发达，可学习先进技术
- **灾难期：**大坍缩发生的时期，环境危险但能目睹事件真相
- **衰败期：**文明衰落后的时期，废墟遍布，适合探索和寻宝
- **变异期：**因时间悖论产生的异常时间线，存在各种变异生物和扭曲现象

时空解谜玩法

- **过去与现在联动：**在过去时间线中打开一扇门，现在时间线中对应的门也会打开
- **时间差利用：**利用不同时间线的地形差异，通过切换时间线来穿越障碍
- **与过去的自己协作：**某些谜题需要玩家在过去留下线索，在现在解读；或者反过来
- **时间悖论事件：**改变过去可能引发时间悖论，产生随机的时空异常事件

经典谜题举例

遗迹大门谜题

过去时间线中大门是完好的，但被机关锁住；现在时间线中大门已破损但机关失效。玩家需要在过去找到机关位置并记住密码，回到现在利用破损处进入。

自我协作谜题

玩家需要在过去时间线留下特定的线索或物品，然后在现在时间线找到这些自己留下的线索来解开谜题。形成"自己帮助自己"的因果循环。

断裂桥梁谜题

现在时间线中桥梁已断裂无法通过。玩家需要回到过去，在桥梁完好时放置重物，导致桥梁在现在时间线的特定位置断裂，形成可跳跃的平台。

时间差谜题

利用不同时间线的时间流速差异，在慢速时间线中完成复杂操作，然后切换到快速时间线让效果瞬间生效。

悖论后果系统

- **轻微悖论：**小范围时空扭曲，部分物品位置改变，不影响主线
- **中度悖论：**NPC记忆改变，部分任务失败或出现新任务
- **重度悖论：**整条时间线分支改变，世界格局发生重大变化
- **悖论修复：**可通过时间碎片修复悖论，或接受悖论创造新的时间线分支

时间线剧情分支

- **历史干预：**玩家可以选择干预或不干预某些历史事件，导致不同的现在
- **多时间线叙事：**主线剧情在多条时间线中并行推进，玩家需要拼凑完整真相
- **因果结局：**游戏结局由玩家在各个时间线的选择共同决定，结局数量超过10种

时间织网机制

 **核心创新：**时间不是线性的，而是一张可以编织的网。玩家可以看到时间线的拓扑结构，在分支节点间自由跳跃。

- **时间网可视化：**打开时间地图，可以看到所有已探索的时间线分支，像神经网络一样交织在一起
- **分支创造：**玩家的重要选择会自动创造新的时间线分支，分支数量没有上限

- **节点跳跃：**可以直接跳转到任意已解锁的时间节点，不需要线性回溯
- **并行解谜：**某些谜题需要同时在2-3条时间线中操作才能解开，玩家需要在多条线间切换配合

悖论资源化系统

 反直觉设计：时间悖论不是惩罚，而是可以收集、交易、利用的珍贵资源。

- **主动造悖：**玩家可以主动制造时间悖论来获取"悖论能量"
- **悖论交易：**神秘的"时间商人"会用稀有物品、特殊能力、隐藏剧情来交换悖论能量
- **悖论武器化：**战斗中可以释放悖论能量，造成时空扭曲伤害，让敌人陷入时间循环
- **悖论修复：**悖论积累过多会导致时间崩溃，但可以用时间碎片或特殊道具修复
- **悖论结局：**高悖论状态下会触发隐藏结局，揭示世界的终极真相

时间熵系统

- **熵值积累：**每次时间穿越、创造分支、制造悖论都会增加时间熵
- **混乱效应：**熵值越高，世界越不稳定——NPC行为随机化、地形异常变化、怪物变异增强
- **混沌机遇：**高熵状态下会出现特殊机遇：稀有物品掉落、隐藏区域开启、神秘商人出现
- **熵值管理：**玩家需要在"稳定安全但收益低"和"混乱危险但收益高"之间做策略选择
- **熵能力：**随着熵值提升，玩家会解锁特殊的熵能力，比如概率操控、因果改写

时间克隆玩法

- **时间印记：**玩家可以在任意时间点留下"时间印记"，创造一个过去的自己的克隆体
- **动作回放：**克隆体会精确重复玩家留下印记时的所有动作，包括移动、攻击、使用技能
- **多体协作：**可以同时存在多个时间克隆体，形成"自己的军队"，协同作战或解谜
- **解谜应用：**某些机关需要同时并在多个位置触发，需要多个克隆体配合完成
- **战斗应用：**战斗中召唤克隆体，可以形成连击、包夹、吸引火力等战术
- **熵消耗：**每个克隆体都会持续消耗时间熵，存在时间有限，需要合理规划使用

3.2 星核共鸣系统

星核共鸣系统的核心是**星核人格化与双向共鸣机制**——每颗星球的星核不只是一个能量源，而是拥有独立意识、性格和记忆的智慧存在。玩家不仅能从星核获得操控环境的能力，更能与星核对话、建立关系，甚至影响星核的性格演化。但这种深度共鸣是双向的——星核也会从玩家身上吸收情感和记忆，玩家的身体和心智也会逐渐发生异化。

星核共鸣机制

- **灵能连接**：玩家需要先找到星核位置，通过灵能仪式建立连接
- **共鸣等级**：共鸣分为三个等级，等级越高能力越强，但反噬风险也越大
- **稳定度系统**：共鸣会消耗灵能稳定度，稳定度过低会产生反噬效果
- **冷却恢复**：共鸣结束后需要一段时间冷却，期间无法再次共鸣

星核种类与特性

星核类型	主属性	特色能力
 大地星核	地形操控	创造岩石屏障、改变地形高度、引发地震
 海洋星核	水系操控	召唤水浪、水下呼吸、创造冰墙
 烈焰星核	火系操控	召唤岩浆、制造火焰风暴、熔化金属
 风暴星核	天气操控	召唤雷电、制造龙卷风、改变风向
 生命星核	生命操控	植物快速生长、治愈生物、唤醒沉睡生命
 虚空星核	时空操控	短距离传送、时间减速、创造重力场

共鸣等级详解

- **第一阶段·浅层共鸣**：可使用基础环境能力，稳定度消耗慢，反噬风险低。持续时间约3分钟
- **第二阶段·深层共鸣**：解锁进阶能力，可改变较大范围环境，稳定度消耗中等。持续时间约1分钟
- **第三阶段·完全共鸣**：解锁终极能力，可在短时间内完全掌控星球环境，但稳定度消耗极快，反噬风险极高。持续时间约15秒

环境操控能力

 地形操控

- 升起或降低地面
- 创造岩石平台
- 打开地下通道
- 改变河流走向

 天气操控

- 召唤雷电攻击敌人
- 制造浓雾隐蔽行踪
- 改变风向辅助飞行
- 召唤暴雨熄灭火焰

 生命操控

- 让植物快速生长
- 唤醒沉睡的生物
- 治愈受伤的生物
- 暂时控制生物行动

灵能反噬风险

- **轻度反噬**：视觉扭曲、操作延迟，持续10秒
- **中度反噬**：生命值持续流失、技能冷却加倍，持续30秒
- **重度反噬**：短暂失去意识（玩家角色昏迷）、周围敌人被吸引，持续60秒
- **永久反噬**：多次重度反噬会导致永久属性下降，需特殊道具恢复

反噬应对与恢复

- **稳定药剂**：可临时提升灵能稳定度，延缓反噬发生
- **冥想恢复**：在安全区域冥想可快速恢复灵能稳定度
- **共生辅助**：某些共生生物可帮助吸收反噬伤害
- **时间回溯**：使用时间能力回溯到反噬前的状态

共鸣组合技

风暴潮

水+风组合，召唤巨大海啸席卷战场，造成大范围伤害并击退敌人

熔岩爆发

火+地组合，从地下召唤熔岩柱，造成持续灼烧伤害并改变地形

生命脉冲

生命+虚空组合，释放生命能量脉冲，治愈友军同时伤害敌人

**策略提示：**不同星球的星核属性不同，玩家需要根据星球特性调整战斗策略。多星核共鸣组合是高阶玩法，需要同时与多颗星球建立连接。

星核人格与对话系统

**核心创新：**星核不是死物，而是拥有自我意识的生命体。玩家获得能力的方式不是"升级"，而是"对话与谈判"。

- **星核意识**：每颗星球的星核都有独特的性格——有的温和友善，有的傲慢多疑，有的疯狂混乱
- **共鸣对话**：进入深层共鸣时，玩家会进入星核的意识空间，与星核直接对话
- **谈判获取能力**：星核不会白白给予力量，玩家需要通过对话说服、交易、甚至欺骗来获得能力
- **选择影响性格**：玩家的选择和行为会改变星核的性格，进而改变它给予的能力类型
- **星核记忆**：星核会记住玩家的所有行为，形成独特的"关系档案"，影响后续对话选项

双向共鸣机制

- **双向影响**：不只是玩家从星核获得力量，星核也会从玩家身上吸收情感和记忆

- **星核进化：**随着共鸣加深，星核会逐渐进化，解锁新的能力和意识层次
- **玩家异化：**深度共鸣会让玩家身体发生异化，获得星核的特征——比如大地星核会让皮肤岩石化
- **共鸣代价：**每次使用星核能力，玩家都会失去一点"人性"，需要在力量和自我之间做选择
- **人性系统：**玩家有人性值，人性值越低，能力越强但越容易失控，结局也会不同

星核关系网络

- **星核间关系：**不同星球的星核之间存在复杂的关系——有的是盟友，有的是仇敌，有的是恋人
- **立场选择：**玩家需要在星核间的矛盾中选边站，获得一方的信任可能会失去另一方
- **星核战争：**特定剧情中，星核之间会爆发战争，玩家需要调停或参与其中
- **隐藏星核：**宇宙中存在隐藏的远古星核，它们拥有禁忌的力量，但也极度危险
- **星核结局：**游戏最终结局会根据玩家与所有星核的关系而变化

3.3 生态共生系统

生态共生系统的核心是**生态意识网络与基因融合**机制——整个星球的生态系统不只是生物的集合，而是一个拥有集体意识的巨大网络。玩家不仅能与单个生物建立共生关系，更能接入整个生态系统的意识网络，感知所有生物的情绪和想法，甚至读取生态系统的集体记忆。通过基因融合，玩家还能吸收生物的身体特征，逐渐演化成超越人类的存在。

共生关系建立

- **信任度系统：**每种生物都有信任度，通过喂食、保护、帮助等行为提升
- **共生仪式：**信任度达到满级后，可以进行共生仪式，建立永久共生关系
- **共生槽位：**玩家最多同时拥有3个共生伙伴，可随时切换
- **共生进化：**通过共同战斗和探索，共生关系可以升级，解锁更强能力

共生进化路线

自然进化

保持生物的原始特性，强化基础能力。进化后能力稳定，反噬风险低。

灵能进化

注入灵能改造生物，解锁特殊能力。进化后能力强大，但可能产生副作用。

共生进化

玩家与生物深度融合，共享感知和力量。终极形态，玩家可暂时化身为生物形态。

稀有共生生物

- **时空蝶：**传说级生物，可短暂操控时间流速，极其稀有，仅在时间褶皱附近出现
- **星核兽：**以星核能量为食的生物，可帮助玩家稳定灵能，减少反噬
- **虚空鲸：**巨型生物，可在太空中生存，玩家可骑乘进行星际旅行

- **幻影狐**：可创造幻象迷惑敌人，还能预知短时间内的危险

共生能力体系

共生类型	代表生物	获得能力
飞行类	光翼鸟、星渊蝠	滑翔、短距离飞行、高空侦察
水生类	深潜鲸、荧光鱼	水下呼吸、快速游泳、声呐定位
战斗类	岩甲兽、毒刺蝎	额外护甲、毒素攻击、战斗辅助
感知类	灵能狐、月影狼	灵能感知、夜视、追踪敌人
辅助类	搬运蚁、治疗蝶	物品存储、生命恢复、环境交互

生态平衡系统

- **生态指数**：每颗星球有生态指数，反映生态系统的健康程度
- **玩家影响**：过度捕猎、破坏环境会降低生态指数，保护和恢复会提升
- **生态奖励**：生态指数高的星球会出现稀有生物、隐藏资源、特殊事件
- **生态惩罚**：生态指数过低会导致物种灭绝、环境恶化、敌人变强
- **生态任务**：玩家可以接取生态保护任务，恢复被破坏的生态环境

动态生态事件

🌸 生物迁徙

特定季节会有大规模生物迁徙，玩家可在迁徙路线上获得稀有材料，但也可能遭遇掠食者。

🌋 生态灾难

火山爆发、洪水等自然灾害会改变生态环境，玩家可选择救援生物或趁乱收集资源。

🦠 物种入侵

外来物种入侵会破坏本地生态，玩家可选择帮助驱逐或研究利用，不同选择影响生态走向。

✨ 灵能潮涌

周期性灵能潮涌事件，生物会变得异常活跃，部分生物会发生临时变异，获得特殊能力。

生物多样性收集

- **生物图鉴**：收集星球上的所有生物，完成图鉴可获得特殊奖励
- **生态研究**：研究生物习性和生态关系，解锁新的共生方式和能力
- **保护成就**：保护濒危物种、恢复生态环境可获得特殊称号和外观奖励
- **种子库系统**：收集各种植物种子，可在自己的基地培育特殊植物

生态意识网络

 **核心创新：**不是收集宠物，而是融入整个生态系统的意识网络。玩家成为生态系统的一部分，而不是站在外面的观察者。

- **意识接入：**玩家可以接入星球的生态意识网络，感知所有生物的情绪和想法
- **集体对话：**与整个生态系统的集体意识对话，了解星球的历史和秘密
- **生物视角：**可以"附身"到任意生物身上，以它们的视角体验世界，完成特殊任务
- **生态记忆：**生态系统拥有集体记忆，玩家可以读取这些记忆，看到远古时代的景象
- **意识污染：**过度接入意识网络会被生态意识同化，逐渐失去自我意识

基因融合系统

- **生物特征融合：**玩家可以吸收生物的基因，获得它们的身体特征——比如长出翅膀、鳃、复眼
- **基因重组：**可以将不同生物的基因组合，创造出独特的混合特征
- **身体异化：**随着融合加深，玩家的身体会越来越不像人类，获得强大能力但也会被NPC恐惧
- **基因稳定度：**基因融合有稳定度，稳定度过低会导致基因突变，产生随机效果
- **形态切换：**玩家可以在不同形态间切换，适应不同的环境和战斗

生态进化引导

- **物种进化：**玩家可以通过选择和干预，引导某个物种向特定方向进化
- **创造新物种：**高阶玩法中，玩家可以创造全新的物种，加入生态系统
- **生态位争夺：**新物种会与原有物种争夺生态位，可能导致原有物种灭绝
- **生态演替：**玩家的行为会引发整个生态系统的演替，从简单到复杂，或从复杂到崩溃
- **时间尺度：**可以利用时间褶皱加速进化过程，在短时间内看到数百万年的演化结果

共生体智能进化

- **共生体学习：**共生生物会学习玩家的行为模式，逐渐变得越来越像玩家
- **语言发展：**深度共生后，共生生物会发展出语言能力，可以与玩家复杂交流
- **人格形成：**每个共生体都会发展出独特的人格，有自己的喜好和目标
- **共生体冲突：**多个共生体之间可能产生矛盾和冲突，玩家需要调解
- **共生体结局：**游戏结局中，共生体也会有自己的命运和故事

3.4 战斗与成长系统

战斗系统采用2.5D上帝视角的策略动作模式，与三大创新玩法深度结合。基于Qt Graphics View框架实现，战斗强调策略选择和环境利用，而非单纯的操作技巧。

时间战斗机制

- **时间减速**：消耗灵能进入子弹时间，可在慢动作中规划攻击路线
- **时间回溯**：战斗中可回溯3秒前的状态，有冷却时间限制
- **时间残影**：创造过去的自己的残影，残影会重复玩家3秒前的动作
- **相位闪避**：短暂进入时间褶皱，免疫所有伤害

共鸣战斗应用

- **环境武器化**：利用星核共鸣能力，将环境转化为武器（落石、雷击、植物缠绕等）
- **地形战术**：改变战场地形，创造有利的战斗位置
- **生物召唤**：唤醒沉睡的生物协助战斗，但会消耗大量灵能稳定度
- **元素组合**：不同环境能力可以组合产生更强效果（如雷+水=范围电击）

共生战斗协同

- **连携攻击**：玩家与共生伙伴同时释放技能，触发强力连携技
- **互补战术**：利用共生伙伴的特殊能力弥补玩家的短板
- **进化觉醒**：战斗中积累共鸣值，达到满值可触发共生觉醒，大幅强化能力
- **牺牲机制**：危急时刻可让共生伙伴承受致命伤害，但会大幅降低信任度

成长体系

- **灵能天赋树**：三大分支（时间系、共鸣系、共生系），玩家可自由搭配
- **稳定度成长**：提升灵能稳定度上限，减少反噬风险
- **共生深度**：提升与共生伙伴的羁绊深度，解锁更多共生能力
- **时间碎片收集**：在时间褶皱中收集时间碎片，用于解锁特殊能力和升级

3.5 三大系统联动玩法

时间褶皱、星核共鸣、生态共生三大系统并非独立存在，而是可以相互配合产生强大的联动效果。掌握联动玩法是高阶玩家的必修课。

时间+共鸣联动

  时间共鸣

 回溯修复

在时间褶皱中使用星核共鸣，共鸣效果会随时间线变化而变化。过去时间线的星核能量更纯净，共鸣效果更强但反噬也更严重。

当灵能反噬发生时，可使用时间回溯回到反噬前的状态，相当于"撤销"反噬效果，但会消耗大量时间能量。

时间+共生联动

时空共生

部分稀有生物（如时空蝶）只能在时间褶皱中找到。与时间生物建立共生关系后，可获得操控时间流速的特殊能力。

过去的伙伴

在过去时间线与生物建立共生关系，回到现在后该生物可能已经进化或灭绝。玩家需要在不同时间线管理自己的共生伙伴。

共鸣+共生联动

共鸣增幅

星核共鸣可以强化共生生物的能力。例如使用生命星核共鸣时，共生生物的治愈能力会大幅提升。

反噬分担

共生伙伴可以帮助玩家分担灵能反噬伤害。共生关系越深，分担的伤害越多，但伙伴也会因此受伤。

三系统终极连携

**终极技能·时空审判：**同时激活三大系统，创造一个时间静止的领域，在领域内玩家可以自由操控环境和召唤生物，对敌人进行毁灭性打击。使用后会进入长时间的虚弱状态。

- **触发条件：**时间能量满、灵能稳定度满、共生觉醒值满，三者同时满足
- **持续时间：**10秒绝对时间静止，期间玩家可自由行动
- **冷却时间：**现实时间24小时（游戏内约3天）
- **副作用：**使用后所有系统进入24小时冷却，灵能稳定度降为0

深度联动：时间织网 × 星核对话

**跨时间线的星核关系网：**每个时间线的星核都是独立的个体，它们之间的关系比你想象的更复杂。

- **时间线星核差异：**同一颗星球的星核在不同时间线有完全不同的性格和记忆——繁荣期的星核骄傲自信，灾难期的星核疯狂绝望，衰败期的星核孤独冷漠
- **跨时间线说服：**可以在过去时间线说服年轻的星核，然后在现在时间线获得已经信任你的星核的帮助
- **星核时间悖论：**改变过去的星核会导致现在的星核性格突变，可能从朋友变成敌人，反之亦然

- **星核时间克隆**：可以召唤不同时间线的星核意识同时出现，形成"星核议会"，共同决策
- **时间商人的真相**：神秘的时间商人其实是某个时间线的星核化身，它在收集悖论能量是为了某个秘密目的

深度联动：时间织网 × 生态意识



时空生态演替：你不仅能看到现在的生态，还能看到过去和未来的生态演化。

- **时间生态地图**：打开时间织网，可以看到每个时间节点的生态状态——从原始海洋到森林覆盖，再到沙漠化
- **加速进化实验**：利用时间褶皱加速局部区域的时间流速，在几分钟内看到数百万年的生态演化结果
- **灭绝物种复活**：可以回到过去，将已灭绝的物种带到现在，重新引入生态系统
- **未来生态预警**：可以看到未来的生态灾难，然后回到现在改变历史，避免灾难发生
- **时间生物**：在时间褶皱中存在只存在于时间夹缝中的生物，它们以时间能量为食，捕捉它们可以获得时间能力

深度联动：星核对话 × 生态意识



星核是生态系统的大脑，而你是连接大脑和身体的神经。

- **星核生态网络**：星核通过灵能网络连接所有生物，形成一个巨大的"星球大脑"。玩家可以接入这个网络，成为星核的"手"和"眼"
- **生物星核化**：深度共鸣后，共生生物会逐渐获得星核的特征，变成"星核生物"，拥有强大的力量
- **生态投票**：可以发起"生态公投"，让所有生物集体投票决定某个重大事件——比如是否接纳某个新物种
- **星核情绪传染**：星核的情绪会通过生态网络传染给所有生物——星核愤怒时，整个星球的生物都会变得狂暴
- **生态治愈星核**：如果星核受伤或生病，可以通过恢复生态系统来治愈星核，生物多样性越高，治愈效果越好

终极联动：三位一体



当时间、星球、生命三者完全融合，你将成为超越维度的存在。

- **创世模式**：同时激活三大系统的终极形态，可以创造一个全新的时间分支，在这个分支里你可以设计一颗全新的星球，创造全新的生态系统

- **时间闭环：**可以创建一个完美的时间闭环，让某个区域的生态永远保持在最佳状态——但这会消耗巨量的悖论能量
- **星核化身体：**终极形态下，玩家的身体会完全星核化，同时拥有时间操控、环境操控、基因操控三种能力，成为行走的神祇
- **终极选择：**游戏最终，玩家面临终极选择——是成为新的星核，永远守护这颗星球？还是打破时间循环，让一切走向未知的未来？
- **隐藏结局：**如果玩家在所有时间线、所有星核、所有生物中都建立了完美的关系，会触发"三位一体"真结局，揭示整个宇宙的终极秘密

3.6 任务系统

游戏拥有丰富的任务体系，包括主线任务、支线任务、日常任务和随机事件，为玩家提供多样化的游戏体验。

3.6.1 主线任务

- **剧情驱动：**主线任务围绕"大坍缩"真相和先驱者秘密展开，引导玩家逐步探索整个星系
- **多分支设计：**主线任务存在多个分支，玩家的选择会影响剧情走向和最终结局
- **时间线联动：**部分主线任务需要在不同时间线中完成，过去的选择会影响现在的任务内容
- **章节结构：**主线任务分为5个大章节，每个章节对应一个核心剧情节点

3.6.2 支线任务

- **同伴任务：**每位同伴都有独立的个人任务线，深入挖掘其背景故事
- **势力任务：**为不同势力完成任务，提升声望，解锁特殊奖励
- **星球任务：**每颗星球都有独特的支线任务，展现当地的风土人情
- **隐藏任务：**部分隐藏任务需要特定条件触发，奖励稀有物品和剧情

3.6.3 日常与周常任务

- **日常任务：**每日刷新的简单任务，提供基础资源和经验
- **周常任务：**每周刷新的挑战任务，奖励更丰厚
- **活动任务：**限时活动任务，提供限定奖励

3.6.4 随机事件

- **太空遭遇：**星际旅行中随机触发的事件，如海盗袭击、求救信号、神秘商船等
- **星球事件：**在星球表面随机触发的事件，如生物迁徙、异常天气、遗迹发现等
- **时间异常：**时间褶皱中随机出现的异常事件，可能带来机遇或危险
- **生态事件：**生态平衡变化引发的随机事件，影响整个星球的环境

 **任务设计原则：**所有任务都设计多种完成方式，鼓励玩家发挥创造力，利用三大系统的独特能力解决问题。

3.7 数值系统

游戏拥有完整的数值体系，确保游戏平衡性和成长体验。

3.7.1 核心属性

属性名称	类型	说明
生命值	基础	角色生存能力，降为0则死亡
灵能值	基础	释放技能和使用特殊能力的能量
稳定度	特殊	灵能稳定程度，影响反噬风险
时间熵	特殊	时间扭曲程度，影响世界稳定性
攻击力	战斗	造成伤害的能力
防御力	战斗	减少受到伤害的能力
暴击率	战斗	触发暴击的概率
暴击伤害	战斗	暴击时的伤害倍率

3.7.2 成长曲线

- **等级成长：**采用曲线成长，前期升级快，后期升级慢，确保前期有成就感，后期有挑战
- **属性成长：**每级提升基础属性，不同天赋树有不同的属性加成方向
- **难度成长：**敌人等级随玩家等级动态调整，保持适当的挑战难度

3.7.3 伤害公式

 **基础伤害公式：**最终伤害 = 攻击力 × 技能倍率 × (1 - 防御力 / (防御力 + 100)) × 暴击倍率 × 抗性修正

- **元素伤害：**不同元素伤害有对应的抗性，需根据敌人类型选择合适的元素
- **时间伤害：**时间系技能造成特殊的时间伤害，可减速或冻结敌人
- **灵能伤害：**灵能系技能直接攻击敌人灵能，对灵能类敌人有特效

3.7.4 经济系统

- **主要货币：**星际信用点，用于购买装备、道具、升级等
- **特殊货币：**时间碎片、灵能结晶、生态种子等，用于特殊升级和交易
- **获取途径：**任务奖励、敌人掉落、探索宝箱、资源采集等
- **回收系统：**多余装备可分解为材料，用于打造和升级

3.7.5 经验值系统

- **经验获取：**击杀敌人、完成任务、探索发现等都可获得经验
- **等级上限：**初始等级上限为50级，可通过DLC扩展
- **天赋点数：**每级获得1点天赋点，可自由分配到三大天赋树

3.8 物品与装备系统

游戏拥有丰富的物品和装备系统，为玩家提供多样化的build选择。

3.8.1 装备系统

- **装备部位：**武器、护甲、头盔、手套、靴子、饰品，共6个装备部位
- **装备品质：**普通（白）、优秀（绿）、稀有（蓝）、史诗（紫）、传说（橙）、神话（红）
- **装备属性：**每件装备有基础属性和随机词条，词条数量随品质提升
- **装备套装：**特定装备组合可触发套装效果，提供额外加成

3.8.2 武器类型

近战武器

- 光剑：高攻速，中等伤害
- 重锤：低攻速，高伤害，范围攻击
- 匕首：极高攻速，低伤害，暴击率高

远程武器

- 能量枪：中等攻速，中等伤害
- 狙击枪：低攻速，极高伤害，射程远
- 霰弹枪：近距离高伤害，扩散射击

灵能武器

- 灵能法杖：召唤灵能攻击
- 时间怀表：操控时间流速
- 生命种子：召唤生物协助

3.8.3 道具系统

- **消耗品：**治疗药剂、灵能药剂、稳定药剂、各种增益药剂
- **材料：**打造材料、升级材料、特殊材料等
- **任务物品：**任务相关的特殊物品，不可丢弃
- **收藏品：**稀有收藏品，可兑换特殊奖励

3.8.4 背包管理

- **背包容量：**初始50格，可通过升级和道具扩展
- **分类整理：**支持按类型、品质、等级等方式自动整理
- **快速使用：**可设置快捷栏，快速使用常用道具
- **仓库系统：**每个星球都有仓库，可存放不常用物品

3.8.5 商店与交易

- **常规商店：**出售基础装备和消耗品，定期刷新
- **神秘商人：**随机出现的神秘商人，出售稀有物品
- **时间商人：**用悖论能量交换特殊物品和能力
- **势力商店：**提升势力声望可解锁特殊商品

3.8.6 打造与升级

- **装备打造：**使用材料打造装备，品质随机
- **装备强化：**消耗材料提升装备基础属性
- **词条重铸：**重新随机装备的词条属性
- **宝石镶嵌：**在装备上镶嵌宝石，获得额外属性

3.9 成就系统

游戏拥有丰富的成就系统，奖励玩家的探索和挑战精神。

3.9.1 成就分类

分类	数量	说明
剧情成就	20个	完成主线剧情和重要支线获得
探索成就	30个	探索星球、发现隐藏区域、收集物品获得
战斗成就	25个	击杀敌人、完成挑战、特殊战斗方式获得
系统成就	15个	升级、收集、打造等系统相关成就
隐藏成就	10个	特殊条件触发，不显示具体要求

3.9.2 成就难度

- **普通成就：**正常游戏流程即可获得，占60%

- **困难成就：**需要一定努力才能获得，占30%
- **传说成就：**极具挑战性，只有少数玩家能获得，占10%

3.9.3 成就奖励

- **称号奖励：**部分成就解锁特殊称号，可在角色信息中展示
- **外观奖励：**部分成就解锁特殊外观，如皮肤、涂装等
- **道具奖励：**部分成就奖励稀有道具和材料
- **成就点数：**每个成就都有成就点数，可在排行榜展示



全成就奖励：解锁全部成就的玩家将获得"时空旅人"传说称号和独家纪念外观。

3.10 存档系统

游戏采用加密JSON存档格式，确保存档安全和跨设备同步。

3.10.1 存档方式

- **自动存档：**到达存档点、完成任务、切换场景时自动存档
- **手动存档：**玩家可随时在安全区域手动存档
- **快速存档：**支持快速存档和快速读档，方便挑战高难度内容
- **自动保存间隔：**可设置自动保存间隔，从5分钟到30分钟可选

3.10.2 存档槽位

- **手动存档槽：**10个手动存档槽位，玩家可自由命名
- **自动存档槽：**3个自动存档槽位，循环覆盖
- **快速存档槽：**1个快速存档槽位

3.10.3 存档内容

- **玩家状态：**等级、属性、装备、道具、技能等
- **任务进度：**已完成任务、进行中任务、任务选择等
- **探索进度：**已探索区域、已发现地点、已收集物品等
- **时间线状态：**各时间线的状态、悖论记录、时间熵值等
- **关系状态：**与NPC、星核、生物的关系和好感度
- **系统设置：**游戏设置、按键设置、画面设置等

3.10.4 存档安全

- **加密存储：**使用AES-256加密算法加密存档文件
- **完整性校验：**使用SHA-256哈希校验存档完整性
- **自动备份：**每次存档前自动备份上一个存档，防止存档损坏
- **存档修复：**存档损坏时可尝试从备份恢复

3.11 玩家体验设计

注重玩家体验设计，确保不同类型的玩家都能获得良好的游戏体验。

3.11.1 新手引导

- **教学关卡：**新伊甸星球作为新手教学关卡，逐步引导玩家熟悉基本操作
- **分步教学：**采用分步教学方式，每次只介绍一个新系统，避免信息过载
- **实践学习：**通过实际任务和谜题学习，而非单纯的文字说明
- **提示系统：**遇到困难时提供智能提示，可随时开启或关闭
- **可跳过：**有经验的玩家可跳过新手引导，直接开始游戏



引导设计原则：三大创新系统（时间褶皱、星核共鸣、生态共生）分别在不同星球逐步解锁，让玩家有充分时间消化和理解。

3.11.2 难度系统

难度	推荐	说明
故事模式	剧情党	战斗难度极低，专注剧情体验，谜题提供更多提示
普通模式	大多数玩家	平衡的战斗和探索难度，适合首次游玩
困难模式	核心玩家	敌人更强，资源更少，谜题难度提升
极难模式	挑战玩家	极具挑战性，敌人一击必杀，存档点减少

- **动态难度：**游戏会根据玩家表现动态调整难度，确保适当的挑战性
- **随时调整：**玩家可随时在设置中调整难度，无需重新开始
- **难度奖励：**高难度模式获得更多经验和更好的掉落

3.11.3 多周目系统

- **新游戏+：**通关后可开启新游戏+模式，玩家可选择继承部分或全部进度重新开始游戏
- **难度提升：**每多一周目，敌人等级和难度都会提升
- **新内容解锁：**多周目解锁新的剧情、装备、任务和结局
- **时间线差异：**不同周目可能出现不同的时间线分支和剧情
- **周目继承：**可选择继承哪些内容（等级、装备、技能、收集品等）



多周目特色：每个周目都会有独特的变化，包括NPC行为、敌人配置、隐藏内容等，鼓励玩家多次游玩探索。

3.11.4 可访问性设计

- **色盲模式：**支持红绿色盲、蓝黄色盲、全色盲等多种色盲模式
- **字幕设置：**可调整字幕大小、颜色、背景透明度
- **按键自定义：**支持完全自定义按键和手柄布局
- **难度辅助：**无敌模式、一击必杀、无限灵能等辅助选项
- **文本大小：**可调整UI文本大小，适配不同视力的玩家
- **语音朗读：**支持文本转语音，为视障玩家提供便利

3.11.5 游戏时长

- **主线剧情：**约25-30小时
- **主线+支线：**约40-50小时
- **全收集：**约80-100小时
- **多周目：**每个周目约20-30小时
- **DLC内容：**每个DLC约10-15小时

四、角色系统

4.1 主角设定

姓名：玩家自定义

背景：年轻的星际探险家，拥有稀有的灵能天赋。在"大坍塌"中失去了家人，因此对灾难的真相有着强烈的探求欲。

性格：勇敢、好奇、富有正义感，但有时过于冲动。随着剧情推进，性格会根据玩家选择产生变化。

4.2 主要NPC

姓名	身份	简介
艾拉	飞船AI	主角飞船的智能AI，性格活泼，是玩家的主要助手和吐槽担当
马库斯	老探险家	经验丰富的老探险家，主角的导师，知晓许多关于先驱者的秘密
瑟琳娜	联邦军官	星际联邦的精英军官，起初与主角对立，后成为重要盟友
虚空先知	反派首领	虚空教团的领导者，拥有强大的灵能，是"大坍缩"的关键人物
守望者	先驱者遗族	古老文明的最后守护者，掌握着先驱者文明的终极秘密

4.3 同伴系统

玩家可以招募最多3名同伴加入队伍，每位同伴都有独特的技能和背景故事。

- 同伴技能：**每位同伴拥有专属战斗技能，可在战斗中召唤协助
- 好感度系统：**通过对话和任务提升好感度，解锁个人剧情和专属装备
- 同伴任务：**每位同伴都有独立的个人任务线，深入挖掘其背景故事

4.4 敌人设计

- 太空海盗：**常见敌人，使用各种枪械和近战武器
- 虚空教徒：**掌握灵能的狂热信徒，拥有各种法术攻击
- 异星生物：**各星球的本土生物，有独特的攻击方式
- 机械守卫：**先驱者遗迹的守护者，拥有强大的火力和护盾
- BOSS级敌人：**每个主要区域都有独特的BOSS，拥有多阶段战斗

五、关卡与地图设计

5.1 星系地图

游戏世界由一个包含12颗主要星球的星系组成，每颗星球都是一个精心设计的箱庭式关卡。基于Qt Graphics View框架实现2.5D视角，玩家可以自由选择探索顺序。

星球名称	环境类型	主要内容
新伊甸	温带星球	新手星球，人类殖民地，教学关卡
红岩星	荒漠星球	矿业殖民地，太空海盗据点
冰霜星	冰冻星球	先驱者遗迹，极端环境挑战
丛林星	热带雨林	丰富的生态系统，异星生物多样性
机械星	人工星球	先驱者的巨型机械造物，充满谜题
虚空之眼	神秘区域	游戏最终区域，虚空教团大本营

5.2 关卡设计原则

- 多路径设计：**每个区域都有多种到达目标的路径，鼓励探索
- 垂直探索：**充分利用垂直空间，设计高低差和攀爬点
- 环境叙事：**通过场景细节讲述故事，增强沉浸感
- 难度曲线：**从新手友好到高难度挑战，平滑过渡
- 隐藏内容：**每个区域都有隐藏区域和秘密，奖励探索玩家

5.3 副本设计

游戏包含多个大型副本（先驱者遗迹），是获取高级装备和推进剧情的重要场所。

- 每个副本有独特的主题和机制
- 副本内包含解谜、平台跳跃、战斗等多种玩法
- 副本BOSS拥有独特的战斗机制，需要玩家策略应对
- 副本可重复挑战，有难度选择和评分系统

六、美术风格

6.1 整体风格

游戏采用"2.5D科幻手绘"的美术风格，基于Qt Graphics View框架实现。采用等距视角（45度俯视），在保持精美手绘质感的同时加入科幻元素。整体色调偏冷，以深蓝、紫色、青色为主，营造宇宙的神秘与浩瀚感。

6.2 场景美术

- **太空场景：**绚丽的星云、壮观的小行星带、巨大的空间站
- **星球表面：**每颗星球有独特的视觉风格，从荒漠到冰原，从丛林到机械
- **遗迹设计：**融合几何美学与神秘符号，展现先驱者文明的辉煌
- **光影效果：**基于Qt Graphics View的2D光影系统，通过图层混合和粒子效果营造氛围光影

6.3 角色美术

- **服装设计：**科幻风格的服装和护甲，注重功能性与美观性的结合
- **武器设计：**能量武器、实弹武器、灵能武器各有特色
- **特效设计：**华丽但不刺眼的战斗特效，突出打击感

6.4 UI设计

UI基于Qt Quick/QML框架开发，采用简洁的科幻风格，以半透明玻璃质感为主，配合全息投影效果。界面布局清晰，信息层级分明，支持流畅的动画过渡效果。

七、音效与音乐

7.1 音乐设计

- **主题曲：**宏大的管弦乐与电子合成器结合，营造史诗感
- **探索音乐：**舒缓的环境音乐，配合星球环境特色
- **战斗音乐：**节奏强烈的电子摇滚，突出战斗的紧张感
- **剧情音乐：**根据剧情氛围变化，增强情感表达

7.2 音效设计

- **环境音效：**每颗星球有独特的环境音，增强沉浸感
- **战斗音效：**武器射击、技能释放、受击反馈等，突出打击感
- **UI音效：**简洁的科技感音效，不干扰游戏体验
- **语音：**全剧情语音配音，重要角色由知名声优演绎

7.3 配音阵容（计划）

角色	中文配音	英文配音
主角（男）	待定	待定

主角（女）	待定	待定
艾拉	待定	待定
马库斯	待定	待定
虚空先知	待定	待定

八、商业化设计

8.1 定价策略

版本	价格	包含内容
标准版	\$59.99	游戏本体 + 预购奖励
豪华版	\$79.99	游戏本体 + 数字原声带 + 美术集 + 独家皮肤
终极版	\$99.99	豪华版内容 + 季票（3个DLC）

8.2 DLC规划

- **DLC1**: 新星球 + 新剧情（预计发售后6个月）
- **DLC2**: 新同伴 + 新技能树（预计发售后12个月）
- **DLC3**: 大型资料片，全新区域和剧情（预计发售后18个月）

8.3 社区运营

- **官方社区**: 建立官方论坛、Discord、QQ群等社区渠道
- **开发者日志**: 定期发布开发日志，展示开发进度和新内容
- **玩家反馈**: 建立反馈渠道，积极响应玩家意见和建议
- **社区活动**: 定期举办社区活动，如创意大赛、截图大赛等
- **KOL合作**: 与游戏主播、UP主合作，扩大游戏影响力

8.4 MOD支持

- **MOD工具**: 提供官方MOD开发工具和文档

- **创意工坊：**支持Steam创意工坊，方便玩家分享和下载MOD
- **API接口：**提供丰富的API接口，支持深度MOD开发
- **MOD评选：**定期评选优秀MOD，给予奖励和官方推荐
- **兼容性：**确保MOD之间的兼容性，提供冲突检测工具



MOD支持原则：鼓励玩家创作，延长游戏生命周期。但禁止影响游戏平衡性的MOD。

8.5 长期运营规划

- **首发期（0-3个月）：** Bug修复、性能优化、响应玩家反馈
- **成长期（3-6个月）：** 免费内容更新、社区活动、MOD支持
- **成熟期（6-12个月）：** DLC发布、大型更新、电竞赛事（可选）
- **长尾期（12个月以上）：** 持续维护、偶尔更新、移植新平台

九、技术方案

9.1 引擎与工具

技术类别	技术说明
开发框架	Qt 6.10.1（全部使用原生库，无第三方依赖）
开发IDE	Qt Creator（官方IDE，集成调试、设计、性能分析工具）
编程语言	C++ 17
编译器支持	MinGW-w64 / MSVC / GCC（全平台多编译器支持）
构建系统	CMake（Qt Creator内置支持）
2.5D图形渲染	Qt Graphics View Framework（等距视角箱庭渲染）
图片格式支持	Qt Svg + 常规图片格式（SVG/PNG/JPG等）
音频系统	Qt Multimedia
网络功能	Qt Network
数据存储	加密 JSON + QSettings

UI 框架	Qt Widgets / Qt Quick QML
动画系统	Qt Animation Framework
版本控制	Git（Qt Creator内置Git支持）
项目管理	Jira + Confluence

9.2 核心技术

- **Qt Graphics View**：高性能2D图形渲染框架，支持数千个图形项的高效管理
- **Qt Graphics View（2D渲染）**：高性能2D图形渲染，支持数千个图形项的高效管理和碰撞检测
- **信号与槽机制**：Qt核心特性，实现对象间的高效通信和解耦设计
- **元对象系统**：Qt的核心扩展，提供运行时类型信息、动态属性系统和反射能力
- **Qt Quick / QML**：用于快速构建游戏UI界面，支持流畅的动画和过渡效果

9.3 性能目标

- **PC**：1080p/60fps（推荐配置），720p/30fps（最低配置）
- **分辨率**：支持4K分辨率，但2.5D视角下1080p为最佳体验
- **加载时间**：初始加载 < 10秒，关卡切换 < 2秒
- **内存占用**：最低2GB，推荐4GB

9.4 加密JSON存档实现方案

使用Qt原生库实现加密JSON存档系统，确保存档安全和完整性。

9.4.1 数据格式

- **JSON格式**：使用QJsonDocument和QJsonObject序列化游戏数据
- **数据结构**：分层JSON结构，按系统分类存储数据
- **版本兼容**：包含版本号，支持旧版本存档的迁移和升级

9.4.2 加密算法

- **AES-256加密**：使用QCryptographicHash和QAESEncryption实现AES-256加密
- **密钥派生**：使用PBKDF2算法从密码派生加密密钥
- **盐值**：每个存档文件使用随机盐值，增加破解难度

9.4.3 密钥管理

- **主密钥**：存储在可执行文件中，使用混淆技术保护

- **设备指纹：**结合设备硬件信息生成设备密钥，防止存档复制
- **用户密码：**可选用户自定义密码，进一步增强安全性

9.4.4 完整性校验

- **SHA-256哈希：**使用QCryptographicHash计算存档数据的SHA-256哈希
- **校验流程：**读取存档时先验证哈希，确保数据未被篡改
- **损坏处理：**校验失败时自动从备份恢复，或提示存档损坏

9.4.5 多槽位存档

- **槽位管理：**使用QSettings管理存档槽位元数据
- **自动备份：**每次保存前自动备份上一个版本
- **存档缩略图：**保存游戏截图作为存档预览图

 **安全注意：**所有加密和哈希操作均使用Qt原生库，不依赖第三方加密库。密钥存储采用多层保护机制，防止逆向工程。

9.5 技术架构设计

9.5.0 架构模式选型

项目采用**模块化+插件化混合架构**，结合两者优势，既保证核心系统的性能和稳定性，又提供内容层的灵活性和可扩展性。

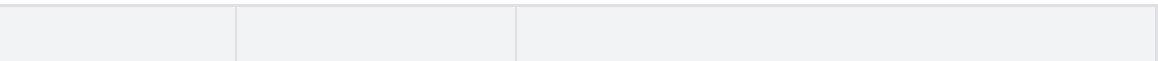
模块化（核心层）

实现方式：编译为动态链接库（.dll/.so/.dylib），启动时加载

- **高性能：**编译时优化，无运行时开销
- **调试方便：**可直接调试，符号完整
- **依赖清晰：**明确的模块依赖关系
- **类型安全：**编译期类型检查

适用：渲染引擎、音频系统、输入系统、数据管理、核心游戏逻辑

整体架构分层



插件化（内容层）

实现方式：运行时动态加载插件（Qt Plugin机制）

- **高度灵活：**无需重新编译主程序即可增减功能
- **支持MOD：**玩家和第三方开发者可创建插件扩展内容
- **热插拔：**开发模式支持运行时重载插件
- **隔离性好：**插件崩溃不影响核心程序

适用：关卡内容、敌人配置、装备道具、扩展玩法、MOD

层级	架构模式	说明
应用层	主程序	游戏主程序，负责启动流程、模块初始化、插件管理
插件层	插件化	各类游戏内容插件、功能扩展插件、MOD插件，运行时动态加载
游戏模块层	模块化	时间褶皱、星核共鸣、生态共生、战斗系统等游戏核心玩法模块
核心模块层	模块化	渲染、音频、输入、资源、数据、UI等基础功能模块
平台层	Qt封装	Qt 6.10.1提供的跨平台能力，封装操作系统差异

 **选型理由：**纯模块化不够灵活，纯插件化性能和调试成本高。混合架构在保证核心系统性能的同时，为内容创作和MOD支持提供了最大的灵活性，非常适合中小型团队的游戏开发。

采用模块化设计，确保代码的可维护性和可扩展性。

9.5.1 模块划分

层级	模块名称	架构模式	主要职责
核心模块层	核心模块	模块化	游戏主循环、事件系统、插件管理器、时间管理、日志系统
	渲染模块	模块化	2D渲染、场景管理、特效系统、粒子系统、UI渲染、瓦片地图
	数据模块	模块化	存档系统、配置管理、数据表加载、加密JSON、本地化
	资源模块	模块化	资源加载卸载、资源缓存、资源打包、热重载支持
	音频模块	模块化	背景音乐、音效、语音、音频混合、3D音效
	输入模块	模块化	键盘鼠标输入、手柄支持、输入映射、输入事件
游戏模块层	玩家系统	模块化	玩家属性、成长系统、天赋系统、装备背包

	战斗系统	模块化	战斗逻辑、伤害计算、AI系统、技能系统
	三大核心系统	模块化	时间褶皱系统、星核共鸣系统、生态共生系统
	任务剧情	模块化	任务系统、对话系统、剧情分支、成就系统
插件层	内容插件	插件化	关卡地图、敌人配置、装备道具、NPC配置
	玩法插件	插件化	小游戏、特殊玩法、自定义系统、扩展机制
	UI插件	插件化	自定义界面、功能面板、MOD UI、皮肤主题
	MOD插件	插件化	第三方MOD、社区创作、完全自定义内容

 **模块间通信：**核心模块之间通过Qt信号槽机制通信，降低耦合；插件通过统一的插件接口与核心交互，不直接依赖具体实现。

9.5.2 核心类设计

- **GameEngine：**游戏引擎主类，管理主循环和各模块
- **SceneManager：**场景管理器，负责场景切换和加载
- **Player：**玩家类，管理玩家状态和能力
- **TimeSystem：**时间褶皱系统，管理时间线和时间操作
- **ResonanceSystem：**星核共鸣系统，管理共鸣和环境操控
- **EcosystemSystem：**生态共生系统，管理生物和共生关系
- **SaveSystem：**存档系统，负责数据的保存和加载
- **ResourceManager：**资源管理器，负责资源的加载和缓存
- **PluginManager：**插件管理器，负责插件发现、加载、卸载、生命周期管理和热重载
- **IPlugin：**插件基类接口，定义所有插件必须实现的统一接口规范
- **ServiceLocator：**服务定位器，为插件提供访问核心系统服务的统一入口
- **EventManager：**事件管理器，实现插件间、插件与核心间的事件驱动通信
- **ExtensionRegistry：**扩展点注册表，管理核心扩展点和插件扩展实现

9.5.3 设计模式

- **单例模式：**管理器类使用单例模式，确保全局唯一访问
- **观察者模式：**事件系统使用观察者模式，实现模块间解耦
- **状态模式：**游戏状态、玩家状态使用状态模式管理
- **工厂模式：**游戏对象创建使用工厂模式
- **策略模式：**AI行为、战斗算法使用策略模式

9.5.4 插件系统设计

基于Qt Plugin机制构建完整的插件系统，支持运行时动态加载、热重载、依赖管理和沙箱隔离，为游戏内容扩展和MOD支持提供基础设施。

插件接口设计

所有插件必须实现统一的IPlugin接口，确保插件与核心程序的解耦：

- **基础接口：**initialize()、shutdown()、getName()、getVersion()、getAuthor()、getDescription()
- **生命周期：**onLoad()加载时调用、onUnload()卸载时调用、onEnable()启用、onDisable()禁用
- **依赖声明：**getDependencies()声明依赖的其他插件，支持版本号约束
- **扩展点：**registerExtensions()向核心注册扩展点，供其他插件调用

插件管理器

PluginManager作为插件系统的核心，负责所有插件的生命周期管理：

- **插件发现：**扫描指定目录，自动发现可用插件
 - **元数据解析：**读取plugin.json元数据，验证插件信息
 - **依赖解析：**拓扑排序，按依赖顺序加载插件
 - **生命周期管理：**加载、初始化、启用、禁用、卸载
- **热重载支持：**开发模式下支持运行时重新加载插件
 - **沙箱隔离：**插件运行在独立上下文，异常不影响核心
 - **权限控制：**细粒度权限控制，限制插件访问范围
 - **插件查询：**提供API查询已加载插件、获取插件实例

插件类型

插件类型	基类接口	说明与示例
内容插件	IContentPlugin	提供游戏内容数据，如关卡地图、敌人配置、装备道具、NPC数据等，纯数据驱动，无需代码

玩法插件	IGameplayPlugin	扩展游戏玩法机制，如新增小游戏、自定义技能系统、特殊游戏模式等
UI插件	IUIPlugin	扩展用户界面，如自定义面板、功能窗口、主题皮肤、HUD组件等
渲染插件	IRenderPlugin	扩展渲染效果，如自定义着色器、后处理效果、粒子特效等
工具插件	IToolPlugin	开发工具类插件，如调试工具、关卡编辑器、数据查看器等
MOD插件	IModPlugin	第三方MOD，拥有最高权限，可修改和替换游戏核心内容

插件元数据

每个插件目录下必须包含plugin.json元数据文件，描述插件基本信息：

- **基本信息：** id、name、version、author、description、website
- **类型声明：** type (content/gameplay/ui/render/tool/mod)
- **依赖声明：** dependencies数组，每个依赖包含id和version范围
- **兼容版本：** game_version，声明兼容的游戏版本
- **权限声明：** permissions数组，声明需要的权限
- **入口点：** main，插件动态库文件名

插件安全与沙箱

 **安全第一：** 插件系统必须有完善的安全机制，防止恶意插件破坏游戏或系统。

- **权限系统：** 细粒度权限控制，插件只能访问声明过的权限范围
- **异常隔离：** 插件异常被捕获，不导致主程序崩溃
- **内存限制：** 可配置插件最大内存使用，防止内存泄漏影响主程序
- **文件系统沙箱：** 插件只能访问指定目录，不能随意读写文件
- **网络隔离：** 默认禁止插件访问网络，需显式声明权限
- **签名验证：** 官方插件支持数字签名验证，防止篡改

插件热重载

开发模式下支持插件热重载，大幅提升开发效率：

- **自动检测：** 监控插件目录，检测到文件变化自动触发重载

- **状态保存：**重载前保存插件状态，重载后恢复状态
- **平滑过渡：**重载过程中游戏不中断，玩家无感知
- **错误回滚：**新版本插件加载失败时，自动回滚到旧版本
- **调试支持：**提供调试接口，可手动触发重载、查看插件状态

插件与核心交互

插件通过核心提供的服务接口与游戏交互，不直接依赖具体实现：

- **服务定位器：**核心提供ServiceLocator，插件通过它获取各种服务
- **事件系统：**插件可监听游戏事件，也可发送自定义事件
- **扩展点机制：**核心定义扩展点，插件可注册扩展实现
- **数据访问：**插件可通过统一API访问游戏数据，如玩家状态、世界数据
- **UI集成：**插件可向游戏UI添加自定义界面元素

9.5.4 架构模式选择

项目采用**模块化为主、插件化为辅**的混合架构模式，兼顾性能稳定性与灵活扩展性。

模块化与插件化对比

维度	模块化（Modular）	插件化（Plugin）
耦合度	低耦合，模块间通过明确接口交互	极低耦合，插件完全独立
加载方式	编译时静态链接 / 启动时加载	运行时动态加载/卸载
性能	好，无额外调用开销	略有开销，接口调用有间接层
开发难度	中等，接口设计要求高	高，需要完整的插件框架
团队协作	好，按模块拆分任务	更好，可完全独立并行开发
扩展性	较好，新增模块需重新编译	极好，支持热插拔扩展
调试难度	低，可直接断点调试	较高，插件问题定位复杂
MOD支持	需要额外设计MOD接口	天然支持，插件即MOD

为什么选择混合架构



核心理念：稳定的基础用模块化保证性能，灵活的玩法用插件化保证扩展性

- **核心系统采用模块化：**渲染引擎、音频系统、输入系统、数据管理、UI框架等基础系统作为静态模块，确保性能最优和稳定性最高，减少运行时不确定因素
- **玩法系统采用插件化设计：**时间褶皱系统、星核共鸣系统、生态共生系统等核心玩法设计为可插拔模块，便于独立开发、测试和迭代调整
- **关卡与内容支持插件加载：**星球关卡、敌人配置、装备道具等内容可通过插件形式动态加载，便于DLC发布和MOD支持
- **充分利用Qt原生能力：**使用Qt的模块化系统（Qt Modules）组织基础功能，使用Qt Plugin框架实现插件化扩展，无需额外造轮子

模块化实施要点

- **模块边界清晰：**每个模块只负责一个明确的功能领域，模块间通过接口通信，不直接依赖实现
- **依赖单向：**上层模块可依赖下层模块，下层模块不依赖上层模块，避免循环依赖
- **接口稳定：**模块接口一旦确定尽量保持稳定，变更需经过评审，确保协作效率
- **独立测试：**每个模块都有独立的单元测试，可单独验证功能正确性

插件化实施要点

- **接口先行：**先定义清晰的插件接口（纯虚函数），再开发具体插件实现
- **版本兼容：**插件接口设计考虑版本兼容性，新增功能通过扩展接口实现，不破坏原有接口
- **沙箱机制：**第三方MOD插件运行在受限环境中，防止恶意代码影响游戏稳定性
- **插件管理器：**统一的插件管理器负责加载、卸载、生命周期管理和依赖解析

9.6 配置文件设计

游戏全部使用 Qt 原生功能自动管理配置，无需手动编写配置文件读写逻辑。默认配置内置在 Qt 资源系统中，用户配置由 QSettings 自动存储在系统配置目录，完全由 Qt 框架负责管理。

9.6.1 配置管理方式

配置类型	管理方式	说明
默认游戏配置	Qt 资源系统	默认配置内置在 .qrc 资源文件中，编译进可执行文件，只读不可修改
用户设置配置	QSettings 自动管理	用户个性化设置由 QSettings 自动存储在系统配置目录，无需手动管理文件路径
游戏存档	QStandardPaths + 加密 JSON	存档文件由 Qt 自动定位到系统标准存档目录，加密存储玩家进度

游戏数据配置	Qt 资源系统	角色、敌人、装备、关卡等游戏数据全部放在 .qrc 资源中，随程序编译
窗口状态	QSettings 自动管理	窗口大小、位置、最大化状态等由 QSettings 自动保存和恢复

9.6.2 Qt 资源系统默认配置

 **game_config.qrc** - 默认配置资源文件

所有默认配置以 JSON 文件形式放在 Qt 资源系统中，编译时自动打包进可执行文件：

game_config.qrc 内容示例：

```
<RCC>
&nbsp;&nbsp;&nbsp;<qresource prefix="/config">
&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;<file>game.json</file>
&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;<file>characters.json</file>
&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;<file>enemies.json</file>
&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;<file>items.json</file>
&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;<file>levels.json</file>
&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;<file>starcore.json</file>
&nbsp;&nbsp;&nbsp;</qresource>
</RCC>
```

使用时直接通过资源路径访问：

```
QFile file(":/config/game.json");
Qt 自动处理资源的读取和缓存，无需手动管理文件路径。
```

9.6.3 QSettings 用户配置

 **QSettings** - 用户配置自动管理

用户设置全部使用 QSettings 管理，Qt 自动处理存储位置和格式：

- **自动存储位置：** Windows 存在注册表，Linux/macOS 存在系统配置目录，无需手动指定路径
- **自动格式处理：** Qt 自动处理配置的读写和序列化，支持字符串、数字、布尔值等
- **自动分组管理：** 使用 beginGroup/endGroup 对配置分组，结构清晰

- **自动版本兼容：**Qt 自动处理配置的版本迁移和兼容性

使用示例：

```
QSettings settings("StarRetrospect", "Game");
settings.setValue("graphics/fullscreen", true);

bool fullscreen = settings.value("graphics/fullscreen", false).toBool();
```

完全由 Qt 自动管理，无需编写任何文件读写代码。

9.6.4 存档文件管理

- **自动路径定位：**使用 QStandardPaths::writableLocation 自动获取系统标准存档目录
- **自动目录创建：**Qt 自动创建存档目录，无需手动检查和创建
- **加密存储：**存档内容使用简单加密，由 Qt 的 QCryptographicHash 辅助处理
- **自动备份：**保存前自动备份上一版存档，防止损坏
- **跨平台兼容：**Qt 自动处理不同平台的路径和格式差异

9.6.5 配置加载机制

配置加载完全由 Qt 自动处理：

- **默认配置：**从 Qt 资源系统读取，程序启动时自动加载
- **用户配置：**QSettings 自动读取，首次运行使用默认值
- **配置覆盖：**用户配置自动覆盖默认配置，由 QSettings 的 value() 函数处理
- **热加载：**开发模式下可重新加载资源，修改配置后立即生效
- **错误处理：**配置损坏时自动使用默认值，Qt 自动处理异常情况

9.7 资源管理方案

游戏全部使用 Qt 资源系统自动管理资源，无需手动编写资源加载和缓存逻辑。所有资源在编译时自动打包进可执行文件，由 Qt 框架负责加载、缓存和释放，开发者只需通过资源路径访问即可。

9.7.1 资源类型

资源类型	格式	说明
图片资源	PNG, JPG, SVG	角色精灵、场景背景、UI界面、图标等，Qt 自动支持多种格式
音频资源	MP3, WAV, OGG	背景音乐、音效、配音等，Qt Multimedia 自动处理

字体资源	TTF, OTF	游戏字体，通过 QFontDatabase::addApplicationFont 加载
数据配置	JSON	关卡、角色、装备等游戏数据，全部放在资源系统中
动画配置	JSON	角色动画、特效动画参数，Qt 自动读取解析
文本资源	JSON, QM	游戏文本、多语言翻译，Qt 国际化系统自动管理
样式表	QSS	UI 样式表，Qt 自动加载应用
着色器	GLSL	特效着色器，通过资源路径直接加载

9.7.2 Qt 资源系统结构

 **resources.qrc** - Qt 资源文件

所有资源通过 .qrc 文件组织，编译时自动打包进可执行文件：

```
<RCC>
    <qresource prefix="/images">
        <file>images/characters/player.png</file>
        <file>images/ui/menu.png</file>
        <file>images/icons/star.png</file>
    </qresource>
    <qresource prefix="/audio">
        <file>audio/music/theme.mp3</file>
        <file>audio/sfx/click.wav</file>
    </qresource>
    <qresource prefix="/fonts">
        <file>fonts/main_font.ttf</file>
    </qresource>
    <qresource prefix="/config">
        <file>config/game.json</file>
        <file>config/levels.json</file>
```

 </qresource>

</RCC>

使用时直接通过资源路径访问，Qt 自动处理加载：

```
QPixmap pixmap(":/images/characters/player.png");
```

```
QSoundEffect sound; sound.setSource(QUrl("qrc:/audio/sfx/click.wav"));
```

完全无需手动管理文件路径和加载逻辑。

9.7.3 资源自动管理特性

- **自动编译打包：** CMake 配置 AUTORCC 后，Qt 自动编译 .qrc 文件，资源打包进可执行文件
- **自动加载缓存：** Qt 自动管理资源的加载和缓存，首次访问时自动加载，后续直接从缓存读取
- **自动内存管理：** Qt 自动管理资源内存，无需手动释放，资源在程序退出时自动清理
- **自动格式支持：** Qt 自动支持多种图片、音频格式，无需手动编解码
- **自动路径处理：** Qt 自动处理跨平台路径差异，资源路径统一使用 :/ 前缀

9.7.4 资源加载策略

核心资源（内置）

- **UI 资源：** 全部内置在 .qrc 中，启动时自动加载
- **核心配置：** 游戏默认配置全部内置，随程序启动
- **常用图标：** 小图标和常用图片全部内置
- **字体文件：** 游戏字体全部内置，自动加载
- **样式表：** UI 样式全部内置，启动时自动应用

大型资源（外部）

- **场景大图：** 大型场景图片作为外部文件，按需加载
- **背景音乐：** 长音频作为外部文件，播放时加载
- **视频资源：** 过场动画等大型视频外部存储
- **DLC 资源：** DLC 内容作为外部资源包，单独分发
- **Mod 资源：** 玩家 Mod 资源外部加载，不修改主程序

9.7.5 外部资源管理

- **自动路径定位：** 使用 QStandardPaths 自动获取资源目录，跨平台兼容
- **自动目录扫描：** Qt 自动扫描资源目录，发现可用资源
- **资源前缀系统：** 使用 qRegisterResourceData 动态加载外部 .rcc 资源包
- **DLC 资源包：** DLC 资源编译为独立的 .rcc 文件，运行时动态加载
- **资源热重载：** 开发模式下可重新加载资源，修改后立即生效

9.7.6 资源管理器简化设计

由于 Qt 资源系统已经自动处理了大部分资源管理工作，资源管理器只需做简单封装：

- **便捷访问接口：**提供统一的资源访问接口，封装资源路径前缀
- **错误处理：**资源加载失败时提供默认占位资源，避免游戏崩溃
- **外部资源加载：**管理外部资源文件和 DLC 资源包的动态加载
- **调试功能：**开发模式下可查看资源列表和加载状态
- **多语言支持：**配合 Qt 国际化系统，自动切换语言资源

核心的资源加载、缓存、内存管理全部由 Qt 自动处理，无需重复造轮子。

9.6 配置文件设计（旧版）

游戏采用 Qt 原生配置管理方案，结合 QSettings 系统自动管理和 JSON 数据配置，无需手动处理文件路径和存储位置，Qt 框架自动完成配置的读写、持久化和跨平台兼容。

9.6.1 配置文件类型

配置类型	文件位置	说明
游戏主配置	config/game.json	游戏全局配置，包括版本号、默认设置、核心参数等
用户配置	user/settings.json	用户个性化设置，包括画面、音效、按键、难度等
存档配置	saves/slot_*.json	游戏存档文件，加密存储玩家进度
角色配置	config/characters/*.json	角色属性、技能、成长曲线等配置
敌人配置	config/enemies/*.json	敌人属性、AI行为、掉落物品等配置
装备配置	config/items/*.json	装备、道具、材料的属性和效果配置
关卡配置	config/levels/*.json	关卡地图、怪物分布、任务配置等
星核配置	config/starcore/*.json	星核属性、共鸣等级、能力参数等配置

9.6.2 主配置文件示例

 **game.json** - 游戏主配置文件示例

```
{
```

```
&&"version": "1.0.0",
&&"build_date": "2026-06-19",
&&"game_title": "星溯旅人",
&&"game_title_en": "Star Retrospect Traveler",
&&"default_settings": {
&&&&"difficulty": "normal",
&&&&"language": "zh_CN",
&&&&"auto_save": true,
&&&&"auto_save_interval": 300
&&},
&&"gameplay": {
&&&&"max_level": 50,
&&&&"max_party_size": 3,
&&&&"max_symbiosis_slots": 3,
&&&&"base_experience_multiplier": 1.0
&&},
&&"performance": {
&&&&"target_fps": 60,
&&&&"min_fps": 30,
&&&&"max_particles": 1000
&&}
}
```

9.6.3 用户配置文件示例



settings.json - 用户设置配置文件示例

```
{
&&"graphics": {
&&&&"resolution": [1920, 1080],
&&&&"fullscreen": true,
&&&&"vsync": true,
```

```
    &&&&"quality": "high",
    &&&&"texture_quality": "high",
    &&&&"shadow_quality": "medium",
    &&&&"particle_quality": "high"
    &&},
    &&"audio": {
        &&&&"master_volume": 0.8,
        &&&&"music_volume": 0.6,
        &&&&"sfx_volume": 0.7,
        &&&&"voice_volume": 0.9,
        &&&&"mute_on_focus_loss": true
    &&},
    &&"controls": {
        &&&&"keyboard_layout": "default",
        &&&&"mouse_sensitivity": 1.0,
        &&&&"invert_y_axis": false,
        &&&&"key_bindings": {
            &&&&&&"move_forward": "W",
            &&&&&&"move_back": "S",
            &&&&&&"move_left": "A",
            &&&&&&"move_right": "D",
            &&&&&&"attack": "LeftMouse",
            &&&&&&"skill_1": "1",
            &&&&&&"skill_2": "2",
            &&&&&&"interact": "E",
            &&&&&&"inventory": "I",
            &&&&&&"map": "M"
        }
    &&},
    &&"gameplay": {
        &&&&"difficulty": "normal",
```

```

&&&&"subtitles": true,
&&&&"subtitle_size": "medium",
&&&&"colorblind_mode": "none",
&&&&"auto_save": true
&&}
}

```

9.6.4 配置加载机制

- **加载顺序：**启动时先加载默认配置，再加载用户配置覆盖默认值，最后加载存档配置
- **热加载：**开发模式下支持配置文件热加载，修改后立即生效，无需重启游戏
- **配置校验：**加载配置时进行格式校验和范围检查，非法值使用默认值替代
- **配置备份：**重要配置修改前自动备份，防止配置损坏导致游戏无法启动
- **版本兼容：**配置文件包含版本号，支持不同版本配置的自动迁移和兼容

9.6.5 QSettings系统配置

除了JSON配置文件外，游戏还使用Qt的QSettings系统存储窗口位置、最近文件列表等轻量级配置，这些配置自动存储在系统注册表或配置目录中。

- **窗口状态：**窗口大小、位置、最大化状态等
- **最近文件：**最近打开的存档文件列表
- **开发工具：**调试工具的窗口布局 and 状态
- **更新检查：**上次更新检查时间等

9.7 资源管理方案（旧版）

游戏采用 Qt 资源系统（qrc）自动管理所有资源，资源文件在编译时自动打包进二进制文件，无需手动处理文件路径和加载逻辑，Qt 框架自动完成资源的读取、缓存和释放。

9.7.1 资源类型

资源类型	格式	说明
图片资源	PNG, JPG, SVG	角色精灵、场景背景、UI界面、图标等
音频资源	MP3, WAV, OGG	背景音乐、音效、配音等
字体资源	TTF, OTF	游戏中文字体、英文字体、特殊艺术字体

地图资源	JSON, TMX	关卡地图、碰撞数据、路径点等
动画资源	JSON, 帧序列	角色动画、特效动画、UI动画
粒子资源	JSON	粒子效果配置、特效参数
文本资源	JSON, PO	游戏文本、对话、多语言翻译
着色器	GLSL, QSS	特效着色器、UI样式表

9.7.2 资源目录结构

 **resources/** - 资源根目录结构

```
resources/
├── images/
│   ├── characters/  # 角色图片
│   │   ├── player/  # 主角
│   │   ├── npcs/    # NPC
│   │   └── enemies/  # 敌人
│   ├── environments/  # 场景环境
│   │   ├── planets/  # 星球场景
│   │   └── dungeons/  # 副本场景
│   ├── ui/          # UI界面
│   │   ├── menus/    # 菜单
│   │   ├── hud/      # 抬头显示
│   │   └── icons/    # 图标
│   └── effects/      # 特效图片
├── audio/
│   ├── music/        # 背景音乐
│   ├── sfx/          # 音效
│   └── voice/         # 配音
├── fonts/            # 字体
└── maps/             # 地图配置
```



```
|—— animations/      # 动画配置
|—— particles/        # 粒子配置
|—— shaders/          # 着色器
└—— locales/          # 多语言文本
    |—— zh_CN/
    |—— en_US/
    └—— ja_JP/
```

9.7.3 资源命名规范

- **命名规则：**全部使用小写字母和下划线，禁止使用中文、空格和特殊字符
- **前缀分类：**资源名称使用前缀标识类型，如 img_、snd_、map_、anim_ 等
- **层级命名：**使用下划线分隔层级，如 img_character_player_idle
- **版本标识：**资源版本使用 v1、v2 后缀，如 img_ui_button_v2.png
- **尺寸标识：**多尺寸资源使用尺寸后缀，如 img_icon_32x32.png

9.7.4 资源加载与卸载

加载策略

- **预加载：**启动时预加载核心资源和UI资源
- **场景加载：**进入场景前加载该场景所需的全部资源
- **按需加载：**稀有资源和大型资源在使用时才加载
- **异步加载：**资源加载在后台线程进行，不阻塞主线程
- **加载进度：**显示加载进度条，提示当前加载内容

卸载策略

- **场景卸载：**离开场景时卸载该场景的专属资源
- **引用计数：**使用引用计数管理共享资源，计数为0时卸载
- **LRU缓存：**资源使用LRU缓存策略，内存不足时卸载最久未使用的资源
- **强制回收：**切换大场景时强制进行垃圾回收
- **内存监控：**实时监控内存使用，超过阈值自动卸载

9.7.5 资源打包与压缩

- **Qt资源系统：**使用Qt的qrc资源系统将资源编译进二进制文件
- **图片压缩：**使用PNG无损压缩，根据平台选择合适的压缩级别
- **音频压缩：**背景音乐使用OGG格式，音效使用WAV格式确保音质
- **资源分包：**核心资源打包进主程序，DLC资源作为单独的资源包
- **加密保护：**重要资源文件进行简单加密，防止资源被轻易提取

9.7.6 资源管理器设计

资源管理器是游戏的核心模块之一，负责统一管理所有游戏资源。

- **单例模式：**全局唯一的资源管理器实例
- **类型安全：**模板化的资源获取接口，确保类型安全
- **错误处理：**资源加载失败时提供默认占位资源，避免游戏崩溃
- **调试功能：**开发模式下可查看资源加载状态、内存占用、缓存命中率
- **热重载：**开发模式支持资源热重载，修改资源后立即在游戏中看到效果

9.8 开发规范与细则

为确保代码质量和团队协作效率，项目制定了详细的开发规范。所有开发人员必须严格遵守以下规范，确保代码风格统一、结构清晰、易于维护。

9.8.1 代码规范

命名规范

类型	规范	示例
类名	大驼峰命名法	GameManager, PlayerController
函数名	小驼峰命名法	updatePosition, takeDamage
变量名	小驼峰命名法	healthPoints, currentLevel
成员变量	m_前缀 + 小驼峰	m_health, m_position
常量	全大写 + 下划线	MAX_HEALTH, BASE_DAMAGE
枚举值	全大写 + 下划线	STATE_IDLE, STATE_ATTACK
文件名	小驼峰或下划线	gameManager.h, player_controller.cpp

常量与魔法数字规范禁止魔法数字：代码中禁止直接出现未命名的数字常量，所有数值必须定义为具名常量或枚举常量**定义：**使用 `const / constexpr` 定义常量，放在对应的头文件或专门的常量头文件中**枚举替代：**状态码、类型标识等使用 `enum class` 定义，禁止使用整数常量**配置化：**可调参数通过配置文件或 `QSettings` 管理，不硬编码在代码中**例外情况：**0、1、-1 等基础计数数字，以及明确数学意义的常量（如3.14159）可直接使用格式规范

- **缩进：**使用4个空格缩进，禁止使用Tab字符
- **大括号：**左大括号不换行，右大括号单独一行；if/for/while必须使用大括号，即使只有一行

- **行宽：**每行不超过120个字符，超过时合理换行
- **空行：**函数之间空一行，逻辑块之间空一行，头文件包含后空一行
- **头文件包含：**按顺序排列：Qt头文件 → 系统头文件 → 项目头文件，每组之间空一行
- **指针与引用：***和&紧贴类型名，如 `QString* name, const int& value`

注释规范

- **文件注释：**每个源文件开头必须有文件注释，包含文件名、功能描述、作者、创建日期
- **类注释：**每个类必须有Doxygen格式注释，说明类的功能、用途、注意事项
- **函数注释：**公有函数必须有Doxygen注释，说明功能、参数、返回值、异常
- **复杂逻辑：**复杂算法和关键逻辑必须有行内注释，说明设计思路
- **TODO标记：**未完成的代码使用 `// TODO: 描述 标记`，便于后续追踪

9.8.2 项目目录结构

 **Qt标准项目结构** - 简洁清晰，所有源文件直接放在项目根目录

项目采用简洁的Qt标准目录结构，便于开发和维护。第一阶段所有源文件直接放在根目录，随着项目发展可按需扩展子目录。

代码块

```
1  StarRetrospectTraveler/
2  |— CMakeLists.txt      # CMake主配置文件
3  |— main.cpp            # 程序入口
4  |— *.h / *.cpp         # 所有头文件和源文件（直接放在根目录）
5  |— assets/             # 资源文件目录
6  |   |— images/         # 图片资源
7  |   |— audio/          # 音频资源
8  |   |— fonts/          # 字体资源
9  |   |— config/         # 配置文件
10 |— docs/               # 文档目录
11 |   |— design/         # 设计文档
12 |   |— api/            # API文档
13 |   |— build/          # 构建文档
```

核心文件说明

文件名	说明
-----	----

GameEngine.h/cpp	游戏引擎主类，管理主循环和各子系统
GameManager.h/cpp	游戏管理器，管理游戏状态和核心玩法
SceneManager.h/cpp	场景管理器，负责场景切换和渲染
RenderManager.h/cpp	渲染管理器，基于Qt Graphics View Framework
ResourceManager.h/cpp	资源管理器，负责资源加载和缓存
InputManager.h/cpp	输入管理器，处理键盘和鼠标输入
AudioManager.h/cpp	音频管理器，负责音乐和音效播放
UIManager.h/cpp	UI管理器，负责用户界面
PluginManager.h/cpp	插件管理器，负责插件加载和管理
GameConfig.h/cpp	游戏配置，管理配置文件的读写
Player.h/cpp	玩家类，管理玩家状态和属性
TimeSystem.h/cpp	时间褶皱系统
ResonanceSystem.h/cpp	星核共鸣系统
EcosystemSystem.h/cpp	生态共生系统

 **结构扩展：** 当前为第一阶段简化结构，随着项目规模扩大，可按功能模块拆分子目录（如 core/, game/, ui/, systems/等），保持代码组织清晰。

9.8.3 编译构建流程

CMake构建配置

- **构建系统：** 使用CMake 3.20+作为构建系统
- **Qt版本：** 使用Qt 6.10.1，通过find_package(Qt6 REQUIRED)查找
- **C++标准：** 使用C++17标准，启用所有警告
- **编译选项：** Debug模式启用调试信息和AddressSanitizer，Release模式启用O3优化；MinGW-w64下自动启用静态链接运行库、大目标文件支持等专用优化
- **模块化：** 每个模块作为独立的静态库，主程序链接所有模块

构建命令

创建构建目录

```
mkdir build && cd build
```

```
&
```

```
# 配置项目
```

```
cmake .. -DCMAKE_BUILD_TYPE=Release -DCMAKE_PREFIX_PATH=/path/to/Qt6
```

```
&
```

```
# 编译项目
```

```
cmake --build . -j$(nproc)
```

```
&
```

```
# 运行单元测试
```

```
ctest
```

Qt Creator开发环境

- **IDE推荐：**推荐使用Qt Creator作为主要开发IDE
- **调试器：**使用GDB或LLDB进行调试
- **代码分析：**启用Clang-Tidy和Clazy静态代码分析
- **格式化工具：**使用ClangFormat自动格式化代码，确保风格统一
- **项目配置：**Qt Creator自动识别CMake项目，支持一键配置和导入，无需手动设置
- **构建套件：**可配置多套构建套件（Qt版本+编译器+调试器），支持Debug/Release/Profile等多种构建模式切换
- **Qt Designer：**内置可视化UI设计器，支持拖拽式界面设计，实时预览效果
- **Qt Quick Designer：**QML可视化设计工具，支持拖拽创建Qt Quick界面
- **版本控制集成：**内置Git支持，可直接在IDE中查看diff、提交代码、拉取推送、查看提交历史
- **编译输出：**编译错误和警告直接显示在编辑器中，点击可跳转到对应代码行

9.8.4 调试技巧

Qt调试工具

- **Q_ASSERT：**在关键位置使用断言，开发模式下捕获错误
- **qDebug()：**使用Qt的调试输出，支持Qt类型的格式化输出
- **日志系统：**使用自定义日志系统，分级别输出（Debug/Info/Warning/Error）

游戏内调试

- **调试控制台：**游戏内置调试控制台，可执行调试命令
- **显示碰撞盒：**调试模式下显示碰撞盒和触发区域
- **性能统计：**实时显示FPS、内存占用、渲染耗时等性能数据

- **对象树：**利用Qt对象树机制，自动管理内存，避免内存泄漏
- **信号槽调试：**启用QT_DEBUG_SIGNALS环境变量调试信号槽连接
- **断点调试：**支持条件断点、数据断点、函数断点，可精确控制程序执行流程
- **变量查看：**实时查看局部变量、成员变量，支持表达式求值、内存查看和对象树浏览
- **调用堆栈：**查看完整的函数调用链，点击可直接跳转到对应源码位置
- **线程调试：**支持多线程调试，可切换查看不同线程的状态、堆栈和变量
- **性能分析器：**集成QProfiler，分析CPU占用、内存分配、图形渲染性能瓶颈
- **内存检测：**集成AddressSanitizer，检测内存泄漏、越界访问、使用已释放内存
- **无敌模式：**调试模式下可开启无敌模式，方便测试
- **传送功能：**可直接传送到任意关卡或位置

9.8.5 版本控制规范

Git工作流

- **主分支：**main分支始终保持可发布状态，只有经过测试的稳定代码才能合并
- **开发分支：**develop分支用于日常开发，新功能从develop分支创建
- **功能分支：**每个新功能创建独立的feature/xxx分支，完成后合并回develop
- **修复分支：**Bug修复创建bugfix/xxx分支，修复后合并回develop和main
- **发布分支：**版本发布前创建release/xxx分支，用于发布前的测试和打磨

提交规范

- **提交频率：**小步提交，每个逻辑单元一次提交，避免超大提交
- **提交信息：**使用约定式提交格式：type(scope): description
- **类型说明：**feat（新功能）、fix（修复）、docs（文档）、style（格式）、refactor（重构）、test（测试）、chore（构建/工具）
- **描述要求：**简洁明了，说明做了什么和为什么这么做
- **提交前检查：**提交前必须编译通过、单元测试通过、代码格式化

9.8.6 代码审查规范

- **强制审查：**所有合并到develop和main分支的代码必须经过代码审查
- **审查人：**至少一名资深开发人员审查通过才能合并
- **审查重点：**代码正确性、性能影响、安全性、可读性、可维护性、是否符合规范
- **审查反馈：**审查意见要具体、建设性，指出问题同时给出改进建议
- **审查时限：**普通代码审查24小时内完成，紧急Bug修复优先审查

9.8.7 插件开发规范

为保证插件质量和兼容性，所有插件开发必须遵循以下规范：

插件开发流程

- **需求定义：**明确插件功能、类型、依赖，编写插件设计文档
- **环境搭建：**配置插件开发环境，引入插件SDK和头文件
- **接口实现：**继承对应插件基类，实现所有纯虚接口
- **功能开发：**实现插件核心功能，通过服务接口与核心交互
- **元数据编写：**编写plugin.json，填写完整的插件信息
- **测试验证：**功能测试、兼容性测试、性能测试、安全测试
- **打包发布：**按规范打包插件，提交审核或发布

插件目录结构

每个插件必须遵循统一的目录结构：

- **plugin.json：**插件元数据文件，必填
- **bin/：**插件动态库文件
- **assets/：**插件资源文件（图片、音频、配置等）
- **config/：**插件配置文件
- **data/：**插件数据文件
- **docs/：**插件文档（使用说明、开发文档等）
- **scripts/：**插件脚本文件（如支持脚本）

插件接口规范

 **接口原则：**插件只能通过公开接口与核心交互，禁止直接访问核心内部实现，禁止依赖核心的私有头文件。

- **基类继承：**必须继承对应类型的插件基类，实现所有纯虚函数

- **宏声明：**使用Qt Plugin宏声明插件，确保正确导出
- **服务获取：**通过ServiceLocator获取核心服务，禁止直接引用核心模块
- **事件通信：**通过EventManager发送和接收事件，降低耦合
- **错误处理：**所有接口必须处理异常，禁止异常泄漏到核心
- **日志输出：**使用核心提供的日志接口输出日志，便于调试

插件测试要求

- **功能测试：**所有功能必须经过完整测试，确保符合设计需求
- **兼容性测试：**在支持的游戏版本上测试，确保兼容
- **性能测试：**插件不得明显影响游戏帧率和内存占用
- **安全测试：**检查是否存在安全漏洞，如越权访问、内存泄漏
- **异常测试：**测试异常情况，确保插件崩溃不影响主程序
- **卸载测试：**确保插件可正常卸载，卸载后游戏恢复正常

插件发布规范

- **版本号规范：**遵循语义化版本（主版本.次版本.修订号）
- **打包格式：**统一使用zip格式打包，目录结构符合规范
- **文件命名：**插件包命名格式：{plugin_id}_{version}.zip
- **文档完整：**必须包含使用说明、更新日志、已知问题
- **依赖声明：**完整声明所有依赖，包括依赖的版本范围
- **签名验证：**官方插件必须使用官方私钥签名，防止篡改

插件审核标准

提交到官方平台的插件必须通过以下审核：

- **功能完整：**插件功能完整可用，无明显bug
- **性能达标：**不明显影响游戏性能，内存占用合理
- **安全合规：**无恶意代码，不收集用户隐私，不违反安全规范
- **兼容稳定：**兼容声明的游戏版本，运行稳定不崩溃
- **文档齐全：**有完整的使用说明和开发文档
- **内容合规：**不包含违法违规、侵权、低俗内容

9.9 测试计划

制定完善的测试计划，确保游戏质量。

9.9.1 测试类型

单元测试

- 核心算法测试
- 数据结构测试
- 工具类测试
- 覆盖率目标：80%

集成测试

- 模块间接口测试
- 系统功能测试
- 数据流测试
- 性能基准测试

游戏测试

- 玩法体验测试
- 平衡性测试
- 难度曲线测试
- 剧情连贯性测试

9.9.2 测试阶段

- **Alpha测试**：内部测试，验证核心功能和玩法
- **Beta测试**：小范围外部测试，收集玩家反馈
- **压力测试**：测试极限情况下的性能和稳定性
- **兼容性测试**：测试不同硬件和操作系统的兼容性

9.9.3 测试工具

- **单元测试**：Qt Test Framework
- **性能分析**：Qt Creator Profiler
- **内存检测**：Valgrind / Visual Studio Debugger
- **Bug管理**：Jira
- **自动化测试**：自定义测试脚本

9.9.4 性能优化方案

- **对象池**：频繁创建销毁的对象使用对象池技术
- **资源预加载**：场景切换时预加载必要资源
- **异步加载**：大资源使用异步加载，不阻塞主线程
- **视锥剔除**：只渲染可见区域的对象
- **LOD技术**：远距离对象使用低多边形模型
- **纹理压缩**：使用压缩纹理减少内存占用

十、开发计划

10.1 团队配置

部门	人数	职责
策划	8人	系统设计、关卡设计、数值策划、剧情策划
程序	15人	引擎、战斗、AI、UI、工具、服务器
美术	20人	原画、3D建模、动作、特效、场景、UI美术
音频	3人	音乐、音效、配音导演
测试	6人	功能测试、性能测试、兼容性测试
运营	4人	市场、社区、用户运营

10.2 开发里程碑

第一阶段：原型验证（第1-3个月）

- 核心战斗系统原型
- 箱庭关卡原型
- 美术风格确定
- 技术选型完成

第二阶段：内容制作（第4-12个月）

- 完成主线剧情框架
- 完成3颗主要星球制作
- 完成核心系统开发
- 完成主要角色建模和动作

第三阶段：全面开发（第13-18个月）

- 完成全部星球和内容制作
- 完成所有系统功能
- 开始内部测试
- 配音和音乐制作完成

第四阶段：打磨优化（第19-24个月）

- 全面性能优化
- Bug修复和平衡性调整
- 封闭Beta测试

- 市场宣发启动
- 正式发售准备

10.3 风险评估

风险项	概率	影响	应对措施
开发延期	中	高	预留缓冲期，分阶段交付
性能不达标	中	高	早期性能测试，持续优化
核心人员流失	低	高	知识共享，文档完善
市场竞争	中	中	差异化定位，精准宣发

10.3.1 补充风险评估

风险项	概率	影响	应对措施
玩法创新不被接受	中	高	早期测试验证，提供传统玩法选项
Qt开发经验不足	中	高	招聘有经验的开发者，技术预研
预算超支	中	中	严格的预算管理，分阶段拨款
宣发效果不佳	中	中	提前布局社区运营，KOL合作
内容质量不达标	低	高	严格的质量把控，多轮迭代优化

10.4 详细开发步骤

第一阶段：项目初始化（第1个月）

1. 环境搭建

- 安装 Qt 6.10.1（包含 Qt Creator、CMake、MinGW/MSVC 编译器）
- 配置开发环境变量和 Qt Kits
- 安装 Git 并配置版本控制

- 搭建 CI/CD 自动化构建环境

2. 项目结构搭建

- 创建 CMake 项目，配置 Qt 模块依赖
- 建立目录结构：src、include、assets、tests、docs
- 配置编译选项：Debug/Release 模式、警告级别
- 引入 Qt 模块：Core、Gui、Widgets、Multimedia、Network、Svg 等

3. 基础框架搭建

- 实现游戏主循环（基于 QTimer 或自定义循环）
- 实现场景管理器（基于 QGraphicsScene 或自定义场景系统）
- 实现资源管理器（图片、音频、配置文件的加载与缓存）
- 实现输入管理器（键盘、鼠标、手柄输入封装）

第二阶段：核心引擎开发（第2-4个月）

1. 图形渲染系统

- 基于 Qt Graphics View 实现 2.5D 渲染管线
- 实现图层管理系统（背景、场景、角色、UI、特效）
- 实现视口相机系统（跟随、缩放、震动效果）
- 实现粒子效果系统（基于 QGraphicsItem 动画）

2. 音频系统

- 基于 Qt Multimedia 实现音频管理器
- 实现背景音乐播放与淡入淡出
- 实现音效播放与混音控制
- 实现音频资源池管理

3. 数据系统

- 基于加密 JSON 实现存档系统
- 基于 QSettings 实现配置管理
- 实现游戏数据序列化与反序列化
- 实现配置表加载与解析

加密 JSON 存档方案

- **数据格式：**所有游戏存档使用 JSON 格式存储，便于阅读和调试
- **加密算法：**使用 AES-256 加密算法对存档文件进行加密，防止玩家修改存档

- **密钥管理：**加密密钥存储在可执行文件中，结合设备指纹生成，每个设备密钥不同
- **数据完整性：**使用 SHA-256 哈希校验数据完整性，检测存档是否被篡改
- **多槽位存档：**支持多个存档槽位，自动备份，防止存档损坏

4. UI 系统

- 基于 Qt Widgets 或 QML 实现 UI 框架
- 实现主菜单、设置界面、背包界面等基础 UI
- 实现 UI 动画与过渡效果
- 实现本地化系统（多语言支持）

第三阶段：游戏系统开发（第5-9个月）

1. 时间褶皱系统

- 实现时间线切换机制（场景状态保存与恢复）
- 实现时间流速控制（快进、慢放、暂停）
- 实现时间悖论检测与处理逻辑
- 实现时间碎片收集与使用系统

2. 星核共鸣系统

- 实现共鸣等级与稳定度数值系统
- 实现环境操控能力（地形、天气、生命）
- 实现灵能反噬效果与恢复机制
- 实现共鸣组合技系统

3. 生态共生系统

- 实现共生生物 AI 行为树
- 实现信任度与进化系统
- 实现生态平衡数值系统
- 实现生物图鉴与收集系统

4. 战斗系统

- 实现战斗状态机与伤害计算
- 实现技能系统与冷却管理
- 实现敌人 AI 与行为模式
- 实现战斗反馈与特效系统

第四阶段：内容制作（第10-18个月）

1. 关卡制作

- 制作 12 颗星球的箱庭关卡
- 设计并实现时间褶皱谜题
- 放置敌人、宝箱、NPC 等内容
- 关卡难度曲线调试与平衡

2. 美术资源制作

- 角色、怪物、场景元素的 2D 手绘资源
- UI 图标、界面、特效美术资源
- 过场动画与剧情插画
- 音效与音乐制作

3. 剧情与任务

- 主线剧情编写与任务实现
- 支线任务与随机事件
- 多结局分支实现
- NPC 对话系统

第五阶段：测试与优化（第19-22个月）

1. 功能测试

- 单元测试与集成测试
- 游戏流程完整性测试
- Bug 修复与回归测试
- 兼容性测试（不同操作系统、硬件配置）

2. 性能优化

- 帧率优化与渲染性能调优
- 内存占用优化与资源管理
- 加载速度优化与资源异步加载
- 使用 Qt Creator 性能分析工具定位瓶颈

3. 平衡性调整

- 数值平衡与难度曲线调整
- 玩家体验反馈收集与优化
- UI/UX 体验优化

第六阶段：发布准备（第23-24个月）

1. 打包发布

- 使用 windeployqt/macdeployqt 部署依赖
- 制作安装包与数字签名
- Steam/Epic 平台上架准备
- DRM 与反作弊系统集成（如需要）

2. 宣发准备

- 宣传视频与截图制作
- 商店页面文案与素材准备
- 媒体评测与社区运营
- 首日补丁准备

10.5 开发注意事项

Qt 游戏开发特殊性

 Qt 是为桌面应用设计的框架，而非专业游戏引擎。开发游戏时需要特别注意性能优化和架构设计，避免使用过重的 Widget 体系做游戏渲染。

- **渲染选择**：2.5D 箱庭游戏使用 Qt Graphics View 框架，支持等距视角和多层视差滚动效果
- **主循环**：避免使用 QWidget 的 paintEvent 做主循环，应使用 QTimer 或自定义游戏循环
- **信号槽**：高频调用的逻辑避免使用信号槽，直接函数调用性能更好
- **内存管理**：Qt 对象树机制方便内存管理，但游戏对象需注意生命周期管理

性能优化要点

- **图形性能**
 - 开启 QGraphicsView 的 OpenGL 视口加速
 - 合理设置 QGraphicsItem 的缓存模式（DeviceCoordinateCache）
 - 减少场景中 Item 的数量，使用合并渲染
 - 大图使用压缩纹理格式，减少内存占用
- **内存优化**
 - 实现资源引用计数与自动回收机制
 - 关卡切换时及时释放不需要的资源

- 使用 QPixmapCache 管理图片缓存
- 避免频繁创建销毁 QObject 对象
- **帧率控制**
 - 固定帧率模式：使用 QTimer 控制更新频率（推荐 60fps）
 - 可变帧率模式：根据帧时间调整逻辑更新步长
 - 使用 QElapsedTimer 精确计算帧时间

代码规范与架构

- **架构设计**
 - 采用组件化设计，避免继承过深
 - 游戏逻辑与 Qt 框架解耦，便于移植和测试
 - 使用 MVC 或 ECS 架构管理游戏实体
 - 核心系统使用单例或服务定位器模式
- **代码规范**
 - 遵循 Qt 编码规范与命名约定
 - 合理使用 Qt 容器类（QVector、QHash 等）
 - 关键路径避免隐式共享的写时拷贝开销
 - 使用 Q_ASSERT 做调试期检查

跨平台兼容性

- **平台差异**
 - Windows：优先使用 MSVC 编译器，性能更优
 - macOS：注意 Retina 屏幕的高 DPI 适配
 - Linux：需测试不同发行版的 Qt 版本兼容性
 - 文件路径使用 QDir::separator() 或 "/"，避免硬编码
- **部署注意**
 - 使用官方部署工具打包依赖库
 - 注意第三方插件的部署（如图片格式插件）
 - 测试干净环境下的运行情况
 - 考虑使用静态编译减少依赖文件数量

版本控制与协作

- **Git 规范**
 - 使用 Git Flow 或类似的分支策略
 - 提交信息清晰描述修改内容
 - 大文件（美术资源）考虑使用 Git LFS
 - 定期合并主分支，避免分支差异过大
- **团队协作**
 - 策划使用配置表（CSV/JSON）驱动游戏数值
 - 美术资源命名规范与目录结构统一
 - 使用任务管理工具跟踪开发进度
 - 定期进行代码审查与技术分享

调试与测试

- **调试技巧**
 - 使用 Qt Creator 的 Debugger 进行断点调试
 - 使用 qInstallMessageHandler 自定义日志输出
 - 实现游戏内调试控制台，方便实时调参
 - 使用 GammaRay 工具分析 Qt 对象结构
 - **测试策略**
 - 核心系统编写单元测试（QTest 框架）
 - 实现自动化测试脚本，回归关键流程
 - 性能测试建立基准数据，持续监控
 - 尽早进行玩家测试，收集真实反馈
-

十一、市场分析

11.1 目标市场

- **核心玩家：**喜欢《质量效应》《星空》《无人深空》等科幻RPG的玩家
- **泛用户：**对科幻题材感兴趣，喜欢箱庭探索游戏的玩家
- **地区分布：**北美（40%）、欧洲（30%）、亚洲（20%）、其他（10%）

11.2 竞品分析

竞品	优势	我们的差异化
星空	IP影响力大，内容丰富	更流畅的战斗系统，更精美的画面
无人深空	探索自由度高	更深度的剧情和角色塑造
质量效应	剧情优秀，角色鲜明	更开放的世界，更多探索内容

11.2.1 独立游戏竞品

竞品	优势	我们的差异化
空洞骑士	箱庭设计优秀，探索感强	科幻题材+时间褶皱机制，更丰富的叙事体验
死亡细胞	战斗流畅，Roguelike玩法耐玩	更深度的剧情和角色成长，多周目体验不同
蔚蓝	关卡设计精妙，故事感人	三大创新系统，更丰富的玩法机制
哈迪斯	Roguelike+叙事结合出色	时间线分支叙事，更立体的故事体验



市场定位：我们的目标是成为独立游戏中的精品，主打创新玩法和深度叙事，与3A大作形成差异化竞争。

11.3 销售预期

- 首月销量：20万份
- 首年销量：80万份
- 生命周期总销量：150万份

十二、结语

《星溯旅人》致力于打造一款兼具探索乐趣与叙事深度的科幻RPG游戏。通过精心设计的箱庭关卡、丰富的剧情内容和策略性的战斗体验，为玩家呈现一段难忘的星际冒险之旅。

我们相信，凭借优秀的游戏品质和精准的市场定位，《星溯旅人》有潜力成为科幻RPG领域的又一佳作。

（注：文档部分内容可能由 AI 生成）